



Vivante Programming: VIP Application Development Guide

Document Revision 0.81

31 March 2023

This document is compatible with:

Vivante ACUITY Toolkit 6.18.0 and above

VivanteIDE 5.8.1 and above

Vivante GALVIP Unified Driver 6.4.14 and above

Vivante VIPLite Driver 1.12.0 and above

VERISILICON

LEVEL B: CONFIDENTIAL – DISTRIBUTION RESTRICTED

Legal Notices

COPYRIGHT INFORMATION

This document contains proprietary information of Vivante Corporation and VeriSilicon Holdings Co., Ltd. They reserve the right to make changes to any products herein at any time without notice and do not assume any responsibility or liability arising out of the application or use of any product described herein, except as expressly agreed to in writing by Vivante and/or VeriSilicon; nor does the purchase or use of a product from Vivante or VeriSilicon convey a license under any patent rights, copyrights, trademark rights, or any other of the intellectual property rights of Vivante, VeriSilicon or third parties.

DISCLOSURE/RE-DISTRIBUTION LIMITATIONS

The information contained herein may be directly re-distributed or may be adapted for re-distribution by SoC vendors who have licensed the related Vivante core IP, with the understanding that re-distribution is limited to those customers of the SoC vendor who have active and valid NDA or licenses applicable for the SoC which contains the related Vivante core IP. Otherwise, the information contained herein is not to be used by or disclosed to the third parties without the express written permission of an officer of Vivante Corporation or VeriSilicon Holdings Co., Ltd.

(VeriSilicon Distribution **LEVEL B: CONFIDENTIAL – DISTRIBUTION RESTRICTED**).

Upon request VeriSilicon provides this document in Word format for adaptive inclusion in documentation intended for the SoC vendor's customers. VeriSilicon requests a pre-release copy of the adaptation for review and approval. In the adapted document please identify the technology and features described in this document using the Vivante trademark and include a reference to the copyright owner, for example: "This section describes the Vivante® GPU and contains copyright material disclosed with permission of VeriSilicon®, who has authorized re-distribution of this material restricted to those NDA partners and licensees of *[the SoC vendor]* who are engineering *[SoC vendor's product]* which includes the discussed Vivante IP."

TRADEMARK ACKNOWLEDGMENT

VeriSilicon® and the VeriSilicon logo design are the trademarks or the registered trademarks of VeriSilicon Holdings Co., Ltd. Vivante® is a registered trademark of Vivante Corporation. All other brand and product names may be trademarks of their respective companies.

For our current distributors, sales offices, design resource centers, and product information, visit our web page located at <http://www.verisilicon.com>.

For technical support, please email vivante-support@verisilicon.com.

Vivante and VeriSilicon Proprietary. Copyright © 2023 by Vivante Corporation and VeriSilicon Holdings Co., Ltd. All rights reserved.

Preface

Glossary

Term	Description
GUI	Graphics user interface
JRE	Java Runtime Environment
NBG	Network binary graph
NN	Neural network
node	Instantiated kernel
OS	Operating system
OVX	OpenVX standard
PPU	Parallel processing unit in VIP
TP	Tensor processor
VIP	Vision Image Processing
unified network project	The source project of .export.data exported from the Vivante ACUITY tool, which can be compiled and run on any target.

Conventions

- Hexadecimal numbers are indicated by the prefix "0x" —for example, 0x32CF.
- Binary numbers are indicated by the prefix "0b" —for example, 0b0011.0010.1100.1111
- Code snippets and file paths are given in Consolas font.
- Replaceable terms in directory and file paths are surrounded with <>.
Example: Verisilicon_Tool_Acuity_Toolkit_<version>.tgz
- Replaceable argument values in command lines and APIs use italics all caps.
Example: *SEPARATED_DATABASE*
- In command lines:
 - **[x]** denotes optional items.
 - **{a, b, c}** denotes only one of the options must be chosen.

References

ACUITY Toolkit

- Vivante Programming: ACUITY Toolkit User Guide
- Vivante Programming: ACUITY Operation Mapping and Support

VivanteIDE

- VivanteIDE User Guide
This document is provided as HTML within the tool, and is also available as a separate PDF file.

Unified driver

- Vivante Programming: Driver Development Guide
- Vivante Programming Memo: Quick Start Guide for VIP and 3D Driver Builds
- Vivante Programming: Platform Integration Guide
- Vivante Programming: Driver Debug Functions
- Vivante Programming: Neural Network OVXLIB Operation Support
- Vivante Programming: OVXLIB Reference Guide

VIPLite driver

- Vivante Programming: VIPLite API
- Vivante Programming: VIPLite Driver Porting Guide

VIP Programming

- Vivante Programming: VIP Binary Graph Reference Guide
- Vivante Programming: VIP Binary Graph Format Specification
- Vivante.VIP.A.Overview
- Vivante.VIP.Programming_Framework
- Vivante.VIP.Programming_EVIS2
- Vivante.VIP.DP.Instructions
- Vivante.VIP-VX.Introduction.to.EasyEVIS
- Vivante Programming: VIP Shader Parameter Configuration
- Vivante.VIP.Offline.Compiler.Guide
- Vivante Programming: Creating User-Defined OpenVX Kernels with VivanteIDE

Table of Contents

Legal Notices.....	2
Preface	3
Glossary	3
Conventions	3
References	4
Table of Contents.....	5
List of Figures.....	8
List of Tables.....	9
1 Introduction	10
2 ACUITY Toolkit	12
2.1 ACUITY Toolkit Packages.....	12
2.2 Environment Variables	12
2.3 Basic Workflow	13
2.3.1 Preparation.....	13
2.3.2 Importing a Network Model.....	15
2.3.3 Quantizing Network Data.....	16
2.3.4 Performing Network Inference.....	17
2.3.5 Exporting a Network Model to an OpenVX application	18
2.3.6 Dumping Tensors.....	20
2.3.7 Visualizing Network Model.....	21
2.4 Advanced Features.....	22
2.4.1 Evaluating an ACUITY Network.....	22
2.4.2 Troubleshooting Inference Errors.....	23
2.4.3 Validating a Classification Network	24
2.4.4 Improving Quantization Precision	27
2.4.5 Performing Kullback-Laiber Divergence Quantization	28
2.4.6 Performing Hybrid Quantization.....	29
2.4.7 Exporting Applications with Pre-Processing Layers Added	30
2.4.8 Exporting Applications with Post-Processing Layers Added	33
2.4.9 Pre-processing Images for the Unified Driver Use.....	34
3 VivanteIDE	37
3.1 Using VivanteIDE GUI	37
3.1.1 VivanteIDE Packages.....	37

3.1.2	VivanteIDE License.....	37
3.1.3	Importing an OpenVX Project.....	37
3.1.4	Running an OpenVX Project	38
3.1.5	Profiling a Network	42
3.1.6	Comparing Dump Results.....	47
3.2	Using VivanteIDE Command Line Tool	50
3.2.1	Installing a vSimulator SDK Package	50
3.2.2	Building an OpenVX Project.....	50
3.2.3	Running an OpenVX Project	51
3.2.4	Profiling a Network	52
4	Unified Driver.....	54
4.1	Unified Driver Packages.....	54
4.2	Building the Unified Driver	54
4.2.1	Building the Driver	54
4.2.2	Porting the Driver	56
4.2.3	Integrating Power Management.....	58
4.3	Loading the Unified Driver	59
4.4	Enabling Network Profiling	60
4.5	Running a Network.....	60
4.6	Developing Applications.....	61
4.6.1	Integrating Networks into Applications	61
4.6.2	Using Image/Tensor Handles.....	65
5	Environment Variables.....	74
5.1	Environment Variables	74
5.1.1	Environment Variables for ACUIITY Generated Application Configuration.....	74
5.1.2	Environment Variables for OVXLIB Configuration	75
5.1.3	Environment Variables for OpenVX Unified Driver Configuration	75
5.2	Use Cases.....	87
5.2.1	Disabling OVXLIB Log Printing.....	87
5.2.2	Enabling OVXLIB Log Printing	87
5.2.3	Running a Network Multiple Times	89
5.2.4	Enabling Driver Log Printing	89
5.2.5	Profiling a Network	90
5.2.6	Profiling Layers	91
5.2.7	Profiling Memory Usage.....	92
5.2.8	Specifying Multiple Optimization Options.....	92

5.2.9	Configuring Multi-VIP Scenarios	92
6	VIPLite Driver	94
6.1	VIPLite Driver Packages	94
6.2	Building the VIPLite Driver	94
6.3	Configuring Network Profiling for VIPLite Driver	95
6.3.1	Network Profile	96
6.3.2	Layer Profile	97
6.3.3	Memory Footprint	97
6.3.4	Inference Time.....	98
6.4	Running Networks and Creating Applications.....	98
7	NBInfo.....	99
7.1	Obtaining the Brief Summary of an NBG File	99
7.2	Querying the Memory Overhead of an NBG File	100
7.3	Querying the Input Parameters in an NBG File	101
7.4	Querying the Output Parameters in an NBG File	101
	Document Revision History.....	103

List of Figures

Figure 1. Application Development Based on Vivante VIP Software.....	10
Figure 2. Graphic View of Network 1enet	21
Figure 3. OpenVX Project Imported.....	38
Figure 4. Run Configurations Dialog	39
Figure 5. Arguments Tab of the Run Configurations Dialog.....	40
Figure 6. Runner Tab of the Run Configurations Dialog.....	40
Figure 7. OpenVX Project Execution Results.....	41
Figure 8. Profile Configurations Dialog	42
Figure 9. Profile Sub-Tab of the Profiler Tab.....	43
Figure 10. Tensor Compare Function Entry	48
Figure 11. Tensor Compare Dialog	48
Figure 12. Dump Result Comparison Output	49
Figure 13. Adding an OpenVX Kernel.....	62

List of Tables

Table 1. Vivante VIP Software Components	11
Table 2. Files Required for Model Input	13
Table 3. Import Script Execution Command Parameters	15
Table 4. Quantization Script Execution Command Parameters	16
Table 5. Inference Script Execution Command Parameters	17
Table 6. Export Script Execution Command Parameters	18
Table 7. Dump Script Execution Command Parameters	20
Table 8. Parameter on the Main Sub-tab.....	43
Table 9. Parameters on the Profiler Sub-tab.....	44
Table 10. Handle APIs	65
Table 11. Environment Variables for ACUITY Generated Application Configuration	74
Table 12. Environment Variables for OVXLIB Configuration	75
Table 13. Environment Variables for OpenVX Unified Driver Configuration	75
Table 14. Network/Layer Profile	91
Table 15. Macro Definitions.....	95
Table 16. Key Fields in a Layer Profile.....	97
Table 17. NBInfo Options.....	99

1 Introduction

This document provides a comprehensive process of application development for deployment onto VIP hardware, as shown in [Figure 1](#). The software components shown in figure are described in [Table 1](#).

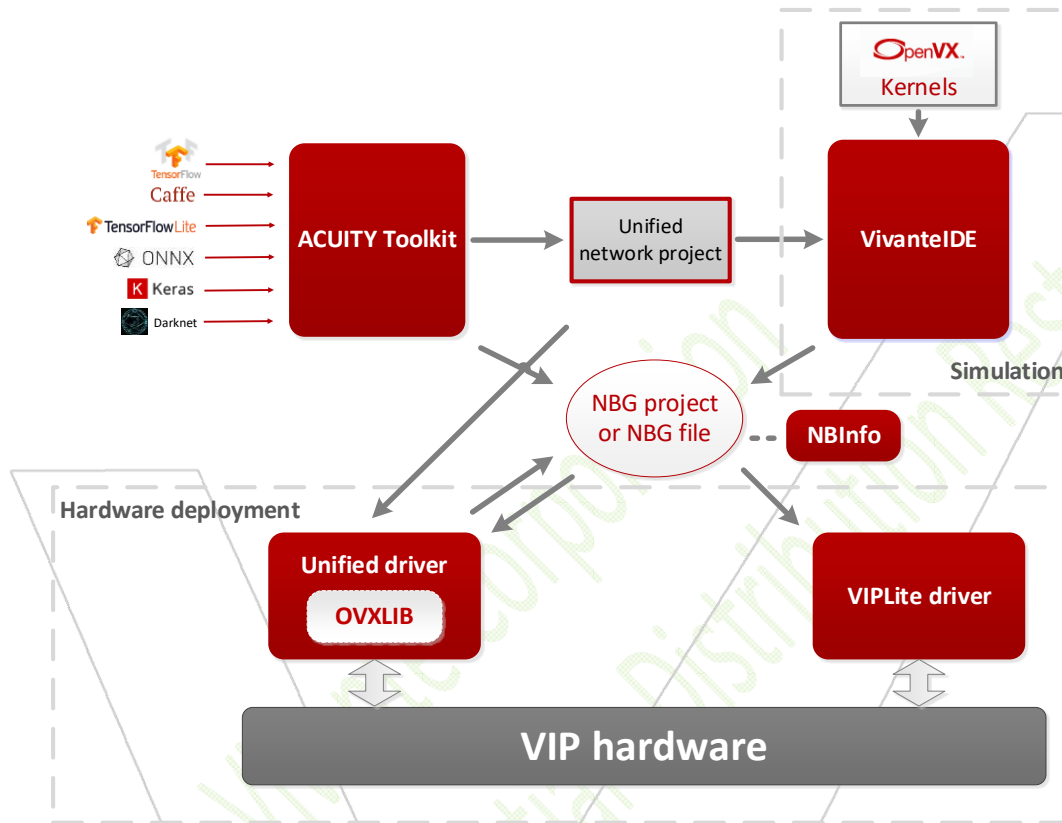


Figure 1. Application Development Based on Vivante VIP Software

Table 1. Vivante VIP Software Components

Component	Description
ACUITY Toolkit	A Linux based toolkit that specializes in processing network models trained by using frameworks such as Caffe and TensorFlow. After translating imported models, ACUITY can optimize models through quantization and inference. ACUITY can export models to unified network projects and NBG projects. - Unified network projects can be run in VivanteIDE or with Vivante GALVIP unified driver through OVXLIB. - NBG projects can be run with Vivante GALVIP unified driver or VIPLite driver.
VivanteIDE	An independent development environment (IDE) for network simulation and result verification. It produces network profile data, including the cycle count and bandwidth. You can also insert OpenVX kernels, such as pre-processing and post-processing nodes, into networks and generate NBG files customized for hardware deployment.
OVXLIB	A library developed based on NN related APIs, utility APIs, and other unified driver APIs. Unified network projects exported from ACUITY Toolkit are OVXLIB based network applications.
Unified driver	Vivante GALVIP unified driver, which loads and runs unified network applications through OVXLIB. It can also be called by OpenVX APIs to run NBG applications and PPU programs.
VIPLite driver	A lightweight driver for RTOS and no-OS devices. It can only run NBG applications.
NBInfo	A tool for analyzing NBG files.
TIM-VX	VeriSilicon Tensor Integration Model for OpenVX (TIM-VX), an open-source software package available on GitHub. It adopts easy-to-use C++ APIs and concepts to support model deployment onto NPUs with Android NN, TensorFlow Lite (TFLite), MLIR, Apache TVM, and other various Machine Learning (ML) frameworks.

2 ACUITY Toolkit

This chapter describes how to develop applications with the ACUITY Toolkit.

For more information about ACUITY Toolkit, see the reference documents listed in the [References](#) section.

2.1 ACUITY Toolkit Packages

ACUITY Toolkit packages include:

- Binary version packages
 - Vivante_acuity_toolkit_binary_<version>_ubuntu16.04.tgz
 - Vivante_acuity_toolkit_binary_<version>_ubuntu18.04.tgz
 - Vivante_acuity_toolkit_binary_<version>_ubuntu20.04.tgz

You can use a binary version directly after decompression. For details, see the *Installing ACUITY Binary Version* section in *Vivante Programming: ACUITY Toolkit User Guide*.

- Wheel version packages
 - Vivante_acuity_toolkit_whl_<version>_ubuntu16.04.tgz
 - Vivante_acuity_toolkit_whl_<version>_ubuntu18.04.tgz
 - Vivante_acuity_toolkit_whl_<version>_ubuntu20.04.tgz

You need to install a wheel version before using it. For details, see the *Installing ACUITY Python Version* section in *Vivante Programming: ACUITY Toolkit User Guide*.

- Example package: acuity_examples_<version>.tgz
This package provides examples of using the ACUITY Toolkit.

Important: Do not decompress ACUITY Toolkit packages on windows.

2.2 Environment Variables

Before using the ACUITY Toolkit, make sure that:

- Environment variable ACUITY_PATH is set to the directory of the ACUITY Toolkit.

```
export ACUITY_PATH=~/.workspace/acuity/acuity-toolkit-binary-<version>/bin
```
- Environment variable CUDA_VISIBLE_DEVICES is set to -1 to turn off the GPU in the case of no GPU accelerator on your local computer.

```
export CUDA_VISIBLE_DEVICES=-1
```


2.3 Basic Workflow

The typical workflow for processing a network model in ACUITY is as follows:

1. Import a network model into ACUITY.
2. (Optional) Quantize the imported network model.
3. Perform inference on the network model.
4. Export the network model to a project.

Note: To facilitate the use of ACUITY, the scripts used in this section uses common command line options.

2.3.1 Preparation

Before importing a network model into ACUITY for further processing, create an input directory using the same name as the imported model file. For example, if the model file is `lenet.tflite`, use `lenet` to name the input directory.

Make sure that the input directory contains the files listed in the following table.

Table 2. Files Required for Model Input

File	Example Used in this Chapter	Description
NN model file	<code>lenet.prototxt</code>	The file type for each supported framework is: • <code>.prototxt</code> for Caffe models • <code>.pb</code> for TensorFlow models • <code>.tflite</code> for TFLite models • <code>.pt</code> for PyTorch models • <code>.h5</code> for Keras models • <code>.onnx</code> for ONNX models • <code>.cfg</code> for Darknet models Important: As the document is updated less frequently than the script, you must always check the supported types of NN frameworks from the received script. For the supported versions of these frameworks, see the <i>Features</i> section in <i>Vivante Programming: ACUITY Toolkit User Guide</i>.
Input images	<code>0.jpg</code>	The input images listed in the dataset file.
Dataset file	<code>dataset.txt</code> , which contains the file path of <code>0.jpg</code>	The supported dataset file types include H5FS, NPY, SQLite, TEXT, and LMDB. For details, see the <i>Dataset Formats</i> section in <i>Vivante Programming: ACUITY Toolkit User Guide</i>.
Mean value file	<code>channel_mean_value.txt</code>	A reference file for you to modify the mean values in the generated <code>inputmeta</code> file.

File	Example Used in this Chapter	Description
Model weight file	N/A	This file is required only for Caffe and Darknet models. - For Caffe models, this file is optional and the file type is <code>.caffemodel</code>. - For Darknet models, this file is mandatory and the file type is <code>.weight</code>.
Model input and output description file	N/A	The file name is <code>inputs_output.txt</code>. This file is required only for TensorFlow, PyTorch, and Keras models.
Inputmeta file	N/A	The file is optional. Its name is in the <code><modelFile_name>_inputmeta.yml</code> format. For details, see the <i>Example: Pre-processing Setups</i> section in <i>Vivante Programming: ACUITY Toolkit User Guide</i>.
Post-processing file	N/A	This file is optional. Its name is in the <code><modelFile_name>_postprocess_file.yml</code> format. For details, see the <i>Example: Post-processing Setups</i> section in <i>Vivante Programming: ACUITY Toolkit User Guide</i>.

Note: The scripts to be run in the following sections must be placed in the parent directory of the created input directory, as shown in the following example:

```

├── lenet
│   ├── 0.jpg
│   ├── channel_mean_value.txt
│   ├── dataset.txt
│   └── lenet.prototxt
├── pegasus_export_ovx.sh
├── pegasus_dump.sh
├── pegasus_import.sh
├── pegasus_inference.sh
└── pegasus_quantize.sh

```

2.3.2 Importing a Network Model

To facilitate NN manipulations such as quantization, inference, dump, and export in ACUITY, you need to import an NN model of a high-level framework to ACUITY. During import, the model is converted into an internal format of ACUITY. For supported NN frameworks, see [Section 2.3.1, Preparation](#).

Procedure

To import a network model, run the import script as follows:

```
./pegasus_import.sh MODEL_DIR
```

The parameter in the command is described in the following table.

Table 3. Import Script Execution Command Parameters

Parameter	Description
MODEL_DIR	The directory that contains the input files, created in Section 2.3.1, Preparation .

Output

In the input directory, a successful execution generates the following files:

- `<MODEL_DIR>.json`: the structure file of the ACUITY network.
- `<MODEL_DIR>.data`: the coefficient data file of the ACUITY network.
This file contains the weight, bias, and other information.
- `<MODEL_DIR>_inputmeta.yml`: the inputmeta file of the ACUITY network.
It is generated only when the input file directory does not contain such file.
- `<MODEL_DIR>_postprocess_file.yml`: the post-processing file of the ACUITY network.
It is generated only when the input file directory does not contain such file.

Note: You need to modify the settings in the inputmeta file and the pro-processing file before subsequent NN manipulations. For details, see the *Example: Pre-processing Setups* and *Example: Post-processing Setups* sections in *Vivante Programming: ACUITY Toolkit User Guide*.

Example

To import the example network model, run the `./pegasus_import.sh lenet/` command.

This command generates the following files in the `lenet` directory: `lenet.json`, `lenet.data`, and `lenet_inputmeta.yml`, and `lenet_postprocess_file.yml`.

In this example, according to the `channel_mean_value.txt` file, you need to set `mean=0` and `scale=1.0/256.0=0.00390625` as follows:

```
preprocess:
reverse_channel: true
  mean:
    - 0
  scale: 0.00390625
```

2.3.3 Quantizing Network Data

After importing a network model, you can quantize it with a specific data type.

For TensorFlow and TensorFlow Lite models, if the original model is a quantized model, quantization in ACUITY is not allowed. Otherwise, errors may occur.

Procedure

To quantize network data, run the quantization script as follows:

```
./pegasus_quantize.sh MODEL_DIR QUANTIZED
```

The command parameters are described in the following table.

Table 4. Quantization Script Execution Command Parameters

Parameter	Description
MODEL_DIR	The directory that contains the input files, created in Section 2.3.1, Preparation .
QUANTIZED	The quantization data type. The available types are uint8, int16, bf16, and pcq. Important: As the document is updated less frequently than the script, you must always check the latest supported data types from the received script.

Output

In the input file directory, a successful execution generates a file `<MODEL_DIR>_<QUANTIZED>.quantize`, which records the quantization results of each layer.

Example

To quantize the data of the example network, run the `./pegasus_quantize.sh lenet/ uint8` command.

This command generates a file `lenet_uint8.quantize` in the `lenet` directory. This file contains the maximum value, minimum value, weight, bias, and scale and counts `zero_point` for each input or output layer as follows:

```
# !!!This file disallow TABs!!!
version: 2
quantize_parameters:
  '@input_0:out0':
    qtype: u8
    quantizer: asymmetric_affine
    rounding: rtne
    max_value: 0.99609375
    min_value: 0.0
    scale: 0.00390625
    zero_point: 0
  '@conv1_1:out0':
    qtype: u8
    quantizer: asymmetric_affine
    rounding: rtne
    max_value: 2.21104097366333
    min_value: -2.705533504486084
    scale: 0.019280685111880302
zero_point: 140
```

2.3.4 Performing Network Inference

You can perform inference on an ACUITY network, either non-quantized or and quantized.

Procedure

To perform inference on an ACUITY network, run the inference script as follows:

```
./pegasus_inference.sh MODEL_DIR QUANTIZED
```

The command parameters are described in the following table.

Table 5. Inference Script Execution Command Parameters

Parameter	Description
MODEL_DIR	The directory that contains the input files, created in Section 2.3.1, Preparation .
QUANTIZED	The tensor data type of the ACUITY network. The available types are float, bf16, int16, uint8, and pcq. bf16, int16, uint8, and pcq are quantization data types. Use them when the input networks are quantized networks. Choose one of them by referencing the data type recorded in the .quantize file, which is in the input file directory. Important: As the document is updated less frequently than the script, you must always check the latest supported data types from the received script.

Output

In the input file directory, a successful execution generates a subdirectory `./inf/<MODEL_DIR>_<QUANTIZED>/` which contains the following files:

- A `.tensor` file to store the network input tensors according to the settings in the inputmeta file.
- A `.tensor` file to store the network output tensors.

When the `--iterations` argument in the script is greater than 1, multiple input and output tensor files are generated. One pair of input and output files corresponds to one execution of inference. For details about the argument, see the *Example: Post-processing Setups* section in *Vivante Programming: ACUITY Toolkit User Guide*.

Example

To perform inference on the example network, run the `./pegasus_inference.sh lenet/ uint8` command.

This command generates a subdirectory `./inf/lenet_uint8/` in the `lenet` directory. The input tensor file and the output tensor file in the subdirectory are as follows:

```
lenet_uint8
├── iter_0_input_0_out0_1_1_28_28.tensor
└── iter_0_output_9_out0_1_10.tensor
```

2.3.5 Exporting a Network Model to an OpenVX application

You can export an ACUITY network model from ACUITY to an OpenVX application. The project can run either on VivanteIDE or devices with Vivante VIPs.

Procedure

1. Open and modify the export script.

```
1. cmd="$PEGASUS export ovxlib \
2.   --model          ${NAME}.json \
3.   --model-data     ${NAME}.data \
4.   --dtype          ${TYPE} \
5.   --batch-size     1 \
6.   --model-quantize ${NAME}_${QUANTIZED}.quantize\
7.   --save-fused-graph \
8.   --target-ide-project 'linux64'\
9.   --optimize       ${OPTIMIZED} \
10.  --viv-sdk         ${VIV_SDK} \
11.  --pack-nbg-unify \
12.  --with-input-meta ${NAME}_inputmeta.yml \
13.  --postprocess-file ${NAME}_postprocess_file.yml \
14.  --output-path     ${generate_path}/${NAME}_${QUANTIZED}"
```

In this script, pay attention to the following parameters:

- Line 9: Set --optimize to the hardware configuration name, in the format VIPXXX_PID0XXX.
- Line 10: Set --viv-sdk to the VivanteIDE SDK directory.
For example, --viv-sdk /VivanteIDE{version}/cmdtools.
- Line 11: If --pack-nbg-unify is specified, two projects are generated for the exported application.
 - lenet_uint8: a unified network project
 - lenet_uint8_nbg_unify: a binary graph project

2. Run the export script as follows to export a network model to an OpenVX application:

```
./pegasus_export_ovx.sh MODEL_DIR QUANTIZED
```

The command parameters are described in the following table.

Table 6. Export Script Execution Command Parameters

Parameter	Description
MODEL_DIR	The directory that contains the input files, created in Section 2.3.1, Preparation .
QUANTIZED	The tensor data type of the ACUITY network. The available types are float, bf16, int16, uint8, and pcq. bf16, int16, uint8, and pcq are quantization data types. Use them when the input networks are quantized networks. Choose one of them according to the data type recorded in the .quantize file, which is in the input file directory. Important: As the document is updated less frequently than the script, you must always check the latest supported data types from the received script.

Output

In the input file directory, a successful execution generates a subdirectory that contains the exported application projects as follows:

Note: `<dataType>` is the quantization data type specified in the export script execution command.

- If the input network is a non-quantized network:
 - `./wksp/<MODEL_DIR>_non-quantized` for the unified network project
 - `./wksp/<MODEL_DIR>_non-quantized_nbg_unify` for the NBG project when `--pack-nbg-unify` is specified
- If the input network is a quantized network:
 - `./wksp/<MODEL_DIR>_<dataType>` for the unified network project
 - `./wksp/<MODEL_DIR>_<dataType>_nbg_unify` for the NBG project when `--pack-nbg-unify` is specified

Example

To export the example network model to an OpenVX application in a unified network project, directly run the following command: `./pegasus_export_ovx.sh lenet/ uint8`.

This command generates a subdirectory `./wksp/lenet_uint8/` in the `lenet` directory. The exported unified network project files in the subdirectory are as follows:

```
lenet_uint8
├── Android.mk
├── BUILD
├── lenet_uint8.export.data
├── lenetuint8.2012.vcxproj
├── lenetuint8.vcxproj
├── main.c
├── makefile.linux
├── vnn_global.h
├── vnn_lenetuint8.c
├── vnn_lenetuint8.h
├── vnn_post_process.c
├── vnn_post_process.h
├── vnn_pre_process.c
└── vnn_pre_process.h
```

What to Do Next

You can use the exported application for simulation on VivanteIDE or hardware deployment with the unified driver. For example, you can run the exported the `lenet_uint8_nbg_unify` project on the unified driver by running the following code:

BUILD	main.c	nbg_meta.json	vnn_global.h	vnn_lenetuint8.h
vnn_post_process.h	vnn_pre_process.h			
lenetuint8.vcxproj	makefile.linux	network_binary.nb	vnn_lenetuint8.c	vnn_post_process.c
vnn_pre_process.c				

If you need to run the application on the VIPLite driver, use the `network_binary.nb` file in the `lenet_uint8_nbg_unify` project. For the sample code to run the application on VIPLite, see the `vpm_run` file in the project.

2.3.6 Dumping Tensors

This section describes how to dump the input and output tensors of each layer in an ACUITY network.

Procedure

To dump the input and output tensors of network layer, run the dump script as follows:

```
./pegasus_inference.sh MODEL_DIR QUANTIZED
```

The command parameters are described in the following table.

Table 7. Dump Script Execution Command Parameters

Parameter	Description
MODEL_DIR	The directory that contains the input files, created in Section 2.3.1, Preparation .
QUANTIZED	The tensor data type of the ACUITY network. The available types are <code>float</code>, <code>bf16</code>, <code>int16</code>, <code>uint8</code>, and <code>pcq</code>. <code>bf16</code>, <code>int16</code>, <code>uint8</code>, and <code>pcq</code> are quantization data types. Use them when the input networks are quantized networks. Choose one of them according to the data type recorded in the <code>.quantize</code> file, which is in the input directory. Important: As the document is updated less frequently than the script, you must always check the latest supported data types from the received script.

Output

In the input file directory, a successful execution generates a subdirectory that contains the dumped tensor files as follows:

- If the input network is a non-quantized network, the directory is `./dump/<MODEL_DIR>_non-quantized`.
- If the input network is a quantized network, the directory is `./dump/<MODEL_DIR>_<dataType>`.
`<dataType>` is the quantization data type specified in the dump script execution command.

Each dumped tensor file is named in the following format:

`<LayerName>_<LayerID>_out0_nchw_1_<Channel>_<Height>_<Width>.tensor`

The file data is arranged in the Caffe sequence.

Note: The difference between dump and inference outputs is:

- The dump output includes the data of all layers.
- The inference output includes only the data of the input and output layers and excludes that of the middle layers.

Example

To dump the input and output tensors of each layer in the example network, run the following command: `./pegasus_dump.sh lenet/ uint8`

This command generates a subdirectory `./dump/lenet_uint8/` in the `lenet` directory. The dumped tensor files in the subdirectory are as follows:

```
lenet_uint8
├── conv1_1_acuity_mark_perm_10_out0_nchw_1_28_28_1.tensor
├── conv1_1_out0_nchw_1_20_24_24.tensor
├── conv2_3_out0_nchw_1_50_8_8.tensor
├── input_0_out0_nchw_1_1_28_28.tensor
├── ip1_5_out0_nchw_1_500.tensor
├── ip2_7_out0_nchw_1_10.tensor
├── output_9_out0_nchw_1_10.tensor
├── pool1_2_out0_nchw_1_20_12_12.tensor
├── pool2_4_out0_nchw_1_50_4_4.tensor
├── prob_8_out0_nchw_1_10.tensor
└── relu1_6_out0_nchw_1_500.tensor
```

2.3.7 Visualizing Network Model

This section describes how to visualize an ACUITY network model using Netron.

Procedure

1. Open the [online Netron viewer](#) or the Netron application.
2. Drag and drop `lenet.json` into the UI.

The structure diagram of the network and the properties of each layer are shown on screen, as shown in the following example.

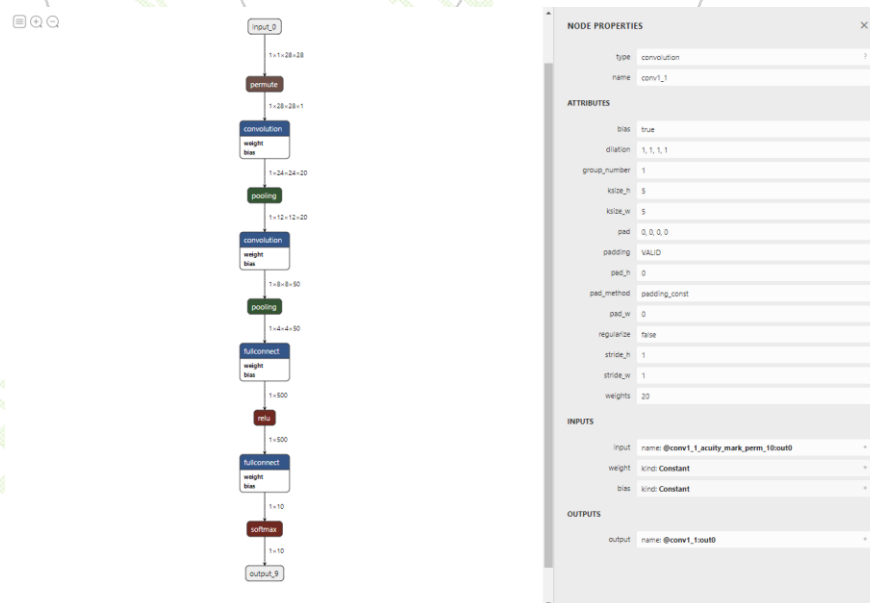


Figure 2. Graphic View of Network lenet

2.4 Advanced Features

2.4.1 Evaluating an ACUITY Network

After a model of a high-level NN framework is imported into ACUITY, the corresponding ACUITY network is generated. The new network needs validation and evaluation before its further NN manipulations in ACUITY.

The evaluation is conducted by precision comparison between the original network and the ACUITY network. During comparison, the inference script is used.

Prerequisites

Make sure:

- The dataset is complete and representative.
- The application or program to calculate the network output precision is prepared, such as the mean average precision (mAP).

The used algorithm must support the full dataset input.

Procedure

1. Calculate the precision of the original network based on the prepared dataset.
2. Calculate the precision of the ACUITY network as follows:
 - a. From the input file directory, open the `<MODEL_DIR>_postprocess_file.yml` file.
 - b. In the file, modify the `postprocess` section, as shown from Line 3 to Line 5 in the following example:

```
1. postprocess:
2.   acuity_postprocs:
3.     dump_results:
4.       file_type: TENSOR
5.       fmt: nchw
```

These parameters define the file type and data format for the network output to be generated in each inference iteration.

- c. Customize the inference script by adding the `--iteration` and `--postprocess` arguments, as shown in Line 6 and Line 7 in the following example:

```
1. cmd="$PEGASUS inference \
2.   --model      ${NAME}.json \
3.   --model-data ${NAME}.data \
4.   --dtype      ${TYPE} \
5.   --device     CPU \
6.   --iterations <numDataset> \
7.   --with-input-meta ${NAME}_inputmeta.yml\
8.   --postprocess-file ${NAME}_postprocess_file.yml"
```

In Line 6, `<numDataset>` is a placeholder. Replace it with the number of inputs in your dataset. This value specifies the number of inference times. One inference processes one input in the dataset file. When the dataset is a TEXT file, the number of inputs is the number of rows in the file.

- d. Run this inference script with the data type set to float.
In the input file directory, a number of output tensor files are generated to store the network outputs. Each of them corresponds to one input in the dataset file.
For information about how to run the inference script, see [Section 2.3.4, Performing Network Inference](#).
- e. Calculate the ACUITY network precision based on all of the network outputs.
3. Compare the precision between the ACUITY network and the original model.
 - If they are quite similar, the ACUITY network is good to use and the evaluation is completed.
 - If the difference is significant, resolve the low-precision issue by following the instructions in [Section 2.4.2, Troubleshooting Inference Errors](#), and then proceed to the next step.
4. Calculate the precision of the ACUITY network again based on the network outputs, and then repeat Step 3.
If low precision persists after performing this step several times, contact [Vivante Technical Support](#).

2.4.2 Troubleshooting Inference Errors

This troubleshooting detects errors by comparing the input and output of the ACUITY network with those of the original model. During detection, the inference and dump scripts are used.

Use this troubleshooting method for a low-precision output after inference of a single input in ACUITY.

Prerequisites

Make sure:

- The ACUITY network is validated, as described in [Section 2.4.1, Evaluating an ACUITY Network](#).
- The input and output data of the original model is retrieved.
The output data of the intermediate layers must also be retrieved. You can use a third party tool to obtain such data.
- The channel, mean, and scale parameters in the inputmeta file are properly set.
For details about the inputmeta file, see the *Example: Pre-processing Setups* section in *Vivante Programming: ACUITY Toolkit User Guide*.

Checking Input Data

1. Run the inference script on the ACUITY network with the float data type.
In the input file directory, the input tensor file is generated. This file stores the network input data in tensor format as specified in the inputmeta file.
For information about how to run the inference script, see [Section 2.3.4, Performing Network Inference](#).
2. Compare the input data with the original network's.
 - If the results are consistent, the input is verified correct.
This indicates that the errors can reside on the intermediate layers or the final output of the ACUITY network. Therefore, check the output data. For details, see the [Checking Output Data](#) section.

Note: Minor inconsistencies are acceptable.

- If the results are inconsistent and the issue cannot be resolved, report the issue to [Vivante Technical Support](#).

For image inputs, inconsistency may come from image decoding. Therefore, convert the dataset into the NPY format, and then proceed to Step 3.

3. Perform inference on the ACUIITY network again with the NPY format input data, and then check the precision of the network output.

If the precision is good, the low precision issue is resolved and the troubleshooting is completed. Otherwise, contact [Vivante Technical Support](#).

Checking Output Data

1. Run the dump script on the ACUIITY network with the float data type.

This dumps output tensors of each layer in the ACUIITY network.

For information about how to run the dump script, see [Section 2.3.6, Dumping Tensors](#).

2. Compare the output data layer by layer between the ACUIITY network and the original model.

If inconsistency is found, contact [Vivante Technical Support](#).

2.4.3 Validating a Classification Network

The inference script can help validate a classification network when the related parameters are configured in both the inputmeta and post-processing files.

Prerequisites

Make sure the ACUIITY network is validated, as described in [Section 2.4.1, Evaluating an ACUIITY Network](#).

Procedure

1. From the input file directory, modify the inputmeta file `<MODEL_DIR>_inputmeta.yml` by adding parameters, as shown from Line 31 to Line 40 in the following example:

```

1. input_meta:
2.   databases:
3.     - path: dataset.txt
4.       type: TEXT
5.       ports:
6.         - lid: input_0
7.           category: image
8.           dtype: float32
9.           sparse: false
10.          tensor_name:
11.            layout: nchw
12.            shape:
13.              - 1
14.              - 1
15.              - 28
16.              - 28
17.          fitting: scale
18.          preprocess:
19.            reverse_channel: true

```

```

20.     mean:
21.     - 0
22.     scale: 0.003921568627451
23.     preproc_node_params:
24.         add_preproc_node: false
25.         preproc_type: IMAGE_RGB
26.         preproc_perm:
27.         - 0
28.         - 1
29.         - 2
30.         - 3
31.     redirect_to_output: false
32. - lid: label_0
33.     redirect_to_output: true
34.     category: unknown
35.     dtype: float32
36.     sparse: false
37.     tensor_name: 'label'
38.     shape:
39.     - 1
40.     - 1

```

Where:

- Line 33: Set `redirect_to_output` to `true` to enable labels given in the dataset file to be directly sent to the network output as an output port.
- Line 32: Assign an ID to the redirect port as `label_0`, which is also an output port of the network.
- Line 38 to Line 40: Set the tensor shape for those redirected labels in the `label_0` port.
- Line 35: Set the `float32` data type for this tensor.

For more details about the inputmeta parameters, see the *Example: Pre-processing Setups* section in *Vivante Programming: ACUITY Toolkit User Guide*.

2. Modify the post-processing file `<MODEL_DIR>_postprocess_file.yml` by adding parameters, as shown from Line 5 to Line 13 in the following example:

```

1. %YAML 1.2
2. ---
3. # "acuity_postprocs" allowed types: "print_topn, classification_classic, detection_validate,
4. dump_results, mute_built_in_actions, classification_validate"
5. postprocess:
6.   acuity_postprocs:
7.     print_topn:
8.       topn: 5
9.     dump_results:
10.      file_type: TENSOR
11.      fmt: nchw
12.     classification_validate:
13.       predictions_tensor: '@output_9:out0'
14.       labels_tensor: '@label_0:out0'
15.     # mute_built_in_actions: true
16.     # python:
17.     #   - file_path: postprocess.py
18.     #   output_tensors:
19.     #     - '@xxxx:out0'
20.   app_postprocs:
21.     - lid: output_9

```

```

21.     postproc_params:
22.         add_postproc_node: false
23.         perm:
24.             - 0
25.             - 1

```

Where:

- Line 6 and Line 7 enable a specified number of top significant network output tensors to be shown on screen.

In this example, top five output tensors are shown on screen.

Note: This feature is useful for classification networks.

- Line 8 to Line 10 configure the dump file type and format.

In this example, the file type is TENSOR and the file format is NCHW.

- Line 11 to Line 13 enable the inference result, the output from the output port output_9, to have a classification label obtained from the output port label_0.

Note: output_9 is read from the ACUITY model file, a .json file. label_0 is read from the inputmeta file.

- Customize the inference script by adding the --iteration and --postprocess-file arguments, as shown in Line 6 and Line 8 in the following example:

```

1. cmd="$PEGASUS inference \
2. --model      ${NAME}.json \
3. --model-data ${NAME}.data \
4. --dtype      ${TYPE} \
5. --device     CPU \
6. --iterations 10 \
7. --with-input-meta ${NAME}_inputmeta.yml\
8. --postprocess-file ${NAME}_postprocess_file.yml"

```

- Run this inference script.

In this example, inference is performed ten times, as specified by the --iteration argument in Step 3. Each inference is applied to an input in the dataset file.

For information about how to run the inference script, see [Section 2.3.4, Performing Network Inference](#).

Results

- In the input file directory, the output tensor file of each inference is generated to contain the classification results in this inference.
- On screen, the top significant classification results of each inference are displayed.

Note: The number of the results displayed after each inference is specified by the topn parameter in the post-processing file.

2.4.4 Improving Quantization Precision

In ACUIITY, quantization precision can be further improved by using the Kullback-Laiber Divergence (KLD) quantization and hybrid quantization.

Prerequisites

Make sure:

- The ACUIITY network is validated, as described in [Section 2.4.1, Evaluating an ACUIITY Network](#).
- The application or program to calculate network output precision for a specific input is prepared, such as Cosine Similarity.

Procedure

1. Perform the KLD quantization on the ACUIITY network with the previously used quantization data type.
For details, see [Section 2.4.5, Performing Kullback-Laiber Divergence Quantization](#).
2. If the KLD quantization precision is below expectation, add the `--MLE` parameter to the quantize command as follows. Using this command to quantize the model takes much time for better model precision.

```
1. cmd="$PEGASUS quantize \
2.   --model      ${NAME}.json \
3.   --model-data  ${NAME}.data \
4.   --algorithm   kl_divergence \
5.   --batch-size  100 \
6.   --divergence-first-quantize-bits 12 \
7.   --device      CPU \
8.   --with-input-meta ${NAME}_inputmeta.yml \
9.   --rebuild \
10.  --MLE \
11.  --model-quantize ${NAME}_${QUANTIZED}.quantize \
12.  --quantizer ${QUANTIZER} \
13.  --qtype ${QUANTIZED} "
```

3. Measure the precision of the quantization result.
 - If the precision is good, you can use the current quantized network for further manipulations in ACUIITY.
 - If the precision needs further improvement, continue with the following steps.
4. Perform hybrid quantization by using the `.quantize` file generated in Step 1.

For details, see [Section 2.4.6, Performing Hybrid Quantization](#).

Note: After hybrid quantization, additional hybrid quantization layers are added to the network. The execution efficiency is decreased if more hybrid quantized layers are added. Therefore, you must consider the trade-off between the precision and efficiency.

5. Repeat from Step 2.

If low precision persists after performing hybrid quantization several times, report this issue to [Vivante Technical Support](#).

2.4.5 Performing Kullback-Laiber Divergence Quantization

ACUITY supports high-accuracy quantization on networks by using the Kullback-Laiber Divergence (KLD) algorithm.

Note: In the KLD algorithm, more input sample images lead to higher quantization accuracy. Therefore, make sure the dataset file contains a large number of images scaling from high to low accuracy samples in various scenes.

Procedure

1. Open and modify the quantization script.

```
1. cmd="$PEGASUS quantize \  
2.   --model      ${NAME}.json \  
3.   --model-data  ${NAME}.data \  
4.   --algorithm   kl_divergence \  
5.   --batch-size  100 \  
6.   --divergence-first-quantize-bits 12 \  
7.   --device      CPU \  
8.   --with-input-meta ${NAME}_inputmeta.yml \  
9.   --rebuild \  
10.  --model-quantize ${NAME}_${QUANTIZED}.quantize \  
11.  --quantizer ${QUANTIZER} \  
12.  --qtype ${QUANTIZED} "
```

Where:

- Line 4: Set --algorithm to kl_divergence.
- Line 6: Set --divergence-first-quantize-bits to 12 or 13, which means 2^{12} or 2^{13} KLD histogram bins.
- Line 5: Set --batch-size to the number of inputs to be fed into the network for quantization.

Set it to a large number by following these rules:

- The larger, the better to gain the best quantization accuracy.
- Make sure the original network supports a batch input. Otherwise, errors can occur.
- This number cannot exceed the input count in the dataset file.
- For a TEXT dataset file, the input count is the number of rows in this file.
- The size of the batch images cannot exceed the maximum size that the system can support.

2. Run the quantization script.

For information about how to run the script and for the output, see [Section 2.3.3, Quantizing Network Data](#).

2.4.6 Performing Hybrid Quantization

Hybrid quantization serves to re-quantize particular layers of a quantized network with data types of higher precisions.

Apply hybrid quantization to a quantized network, if the accuracy can neither meet the requirement nor can be improved by the available quantization methods, such as the Kullback-Laiber Divergence quantization.

Procedure

1. Add the `--compute-entropy` argument to the quantization script.
2. Run the quantization script to perform quantization and generate the following files:
 - `<MODEL_DIR>_<dataType>.quantize`
 - `entropy.txt`: resides in the input file directory and contains the entropy of each layer for this quantization.

The higher the entropy, the lower the quantization accuracy. The range of the entropy values is [0, 1].

3. From the input file directory, open the `<MODEL_DIR>_<dataType>.quantize` file and locate the `customized_quantize_layers` section.

This section provides the suggested layers for re-quantization with the default `dynamic_fixed_point-i16` data type.

4. In the `customized_quantize_layers` section, add, modify, or remove layers for re-quantization, and then save this updated quantization file.

For example:

```
1. customized_quantize_layers:
2. conv2_3: dynamic_fixed_point-i16
```

Where `conv2_3` is a layer to be re-quantized. `dynamic_fixed_point-i16` is its new quantization data type.

To determine the layers for re-quantization, reference the entropies in `entropy.txt`. For high entropy layers, you can use the `float32` type to improve the precision.

The supported data types in this section are `dynamic_fixed_point-i16` and `float32`.

5. Modify the quantization script used previously as follows:
 - Use `--hybrid` to replace the `--rebuild` augment.
6. Run the modified script to perform hybrid quantization.

Results

At a successful execution, in the input file directory:

- A `<MODEL_DIR>.quantize.json` file is generated.
 - This new network file contains data type conversion operations for the re-quantized layers.
- The `<MODEL_DIR>_<dataType>.quantize` file is updated.

What to Do Next

Export this hybrid quantization network to an OpenVX application as follows:

1. From the input file directory, override the `<MODEL_DIR>.json` file with the content of the generated `<MODEL_DIR>.quantize.json` file.

This is because the `.json` file generated by the hybrid quantization has a different name from the file name required for the export script.

2. Export the network with the script, as described in [Section 2.3.5, Exporting a Network Model to an OpenVX application](#).

The exported OpenVX application also contains the data type conversion operations for the re-quantized layers.

2.4.7 Exporting Applications with Pre-Processing Layers Added

With the pre-processing layer added to an OpenVX application, during the execution of the OpenVX application, the input data is directly processed by PPU instead of CPU.

Prerequisites

Make sure the ACUITY network is validated, as described in [Section 2.4.1, Evaluating an ACUITY Network](#).

Procedure

1. From the input file directory, modify the preprocess section in the `<MODEL_DIR>_inputmeta.yml` file, as shown in the following example:

```
1. preprocess:
2.   reverse_channel: false
3.   mean:
4.     - 127.5
5.     - 127.5
6.     - 127.5
7.   scale: 0.0078125
8.   preproc_node_params:
9.     add_preproc_node: true
10.    preproc_type: IMAGE_RGB
11.    preproc_image_size:
12.      - 432
13.      - 432
14.    preproc_crop:
15.      enable_preproc_crop: true
16.      crop_rect:
17.        - 10
18.        - 10
19.        - 234
20.        - 234
21.    preproc_perm:
22.      - 0
23.      - 1
24.      - 2
25.      - 3
```

Where:

- Line 9: `add_preproc_node` must be set to `true` to enable a pre-processing layer to be added to the generated applications.
If `add_preproc_node` is `true`, the pre-processing layer is added to the NBG project during binary graph packing through the ACUITY command `export ovxlib`.
- Line 10: `preproc_type` defines the input type on this layer.
- Line 3 to Line 7: `mean` and `scale` serve to normalize the input data.
- Line 12 to Line 13: the input image size. Line 12 is the width and line 13 is height. If `enable_preproc_crop` is `false`, the input image is resized to the input size of the original model.
- Line 15: `enable_preproc_crop` must be set to `true` if you need to crop an image.
- Line 17 to line 20: the area to be cropped. Line 17 is the start row. Line 18 is the start column. Line 19 is the width of the cropped area. Line 20 is the height of the cropped area. In the example, the input image size is (432, 432), crop rectangular is {10,10,224,224} that likes `img[10:234,10:234]` in Python. Then, resize the cropped image to the input size of the original model.

For more details about the inputmeta parameters, see the *Example: Pre-processing Setups* section in *Vivante Programming: ACUITY Toolkit User Guide*.

2. Run quantization and inference on the ACUITY network with this inputmeta file.

For the quantization script, see [Section 2.3.3, Quantizing Network Data](#).

For the inference script, see [Section 2.3.4, Performing Network Inference](#).

Either inference or quantization results must meet the requirement. To resolve errors, see [Section 2.4.2, Troubleshooting Inference Errors](#) and [Section 2.4.4, Improving Quantization Precision](#).

3. Run the export script, as described in [Section 2.3.5, Exporting a Network Model to an OpenVX application](#).

This generates an OpenVX application with the pre-processing layer added.

Results

Settings in the preprocess section in the inputmeta file are ported into the `vnn_pre_process.c` file, which is generated together with the OpenVX application.

What to Do Next

Use the following sequence to run this OpenVX application on the VivanteIDE or on a device with Vivante VIPs:

1. Set the following environment variable:

```
export VSI_USE_IMAGE_PROCESS=1
```

2. Run the application either on the VivanteIDE or on the device.

Note: Pre-processing is executed on Vivante PPUs. In the VivanteIDE, the PPUs are emulated.

2.4.7.1 Input Types for Pre-processing in OpenVX Applications

The following types of images can be pre-processed within applications that are generated from the export script.

Note: To enable pre-processing within applications, before running the export script, you must configure the used inputmeta file by setting the `add_preproc_node` parameter to `true`, setting `preproc_type` to one of following types, and configuring other pre-processing parameters as described in the *Example: Pre-processing Setups* section in *Vivante Programming: ACUITY Toolkit User Guide*.

- **IMAGE_RGB**
It uses one input port. The pixel data is in packed RGB or packed BGR format.
For example, RGBRGBRGBRGB.....
- **IMAGE_RGB888_PLANAR**
It uses one input port. The R, G, and B channels are recorded in the same input.
The pixel data is in planar RGB or planar BGR format.
For example, RRRR.....GGGG.....BBBB.....
- **IMAGE_RGB888_PLANAR_SEP**
It uses three input ports. Each channel is recorded in a separate input.
The pixel data is in planar RGB or planar BGR format.
- **IMAGE_I420**
It uses three input ports. The first port records the Y value, the second port records the U value, and the third port records the V value.
- **IMAGE_NV12**
It uses two input ports. The first port records the Y value and the second port records the UV value.
- **IMAGE_YUV444**
It uses one input port. The pixel data is in planar YUV format.
For example, YYYY.....UUUU.....VVVV
- **IMAGE_GRAY**
It uses one input port to record only gray pixel values.
Note: Use this format for gray images only.
- **IMAGE_BGRA**
It uses one input port. The pixel data is in packed RGBA or packed BGRA format.
For example, RGBARGBARGBARGBA.....
- **TENSOR**
It uses one input port. The pixel data is in tensor format.

2.4.8 Exporting Applications with Post-Processing Layers Added

This task enables post-processing on Vivante NPUs, either on the parallel processing units (PPU) or on the tensor processors (TP), instead of CPU. With the post-processing layer added to an OpenVX application, the output tensors of this application are converted to the float32 data type for easy access from CPU.

Prerequisites

Make sure quantization and inference of the target ACUITY network are correct and meet the requirement.

To resolve issues, see [Section 2.4.2, Troubleshooting Inference Errors](#) and [Section 2.4.4, Improving Quantization Precision](#).

Procedure

1. From the input file directory, modify the postprocess section in the `<MODEL_DIR>_postprocess_file.yml` file, as shown from Line 8 to Line 12 in the following example:

```

1. %YAML 1.2
2. ---
3. # "acuity_postprocs" allowed types: "classification_validate, print_topn, dump_results, detection_
4. validate, classification_classic, mute_built_in_actions"
5. postprocess:
6.   app_postprocs:
7.     - lid: output_9
8.       postproc_params:
9.         add_postproc_node: true
10.        perm:
11.          - 0
12.          - 1
13.        force_float32: true

```

Where:

- Line 8: `add_postproc_node` must be set to `true` to enable a post-processing layer to be added to the generated application.
The post-processing layer has two functions. One is to convert the output to float32 and the other is to permute.
If `add_postproc_node` is true, the post-processing layer is added in the NBG project during binary graph packing through the ACUITY command `export ovxlib`.
- Line 9 to Line 11: `perm` represents permute. The number of its dimensions must be the same as the network output tensor's. Otherwise, errors can be introduced into the application.
- Line 12: `force_float32` means to force convert the output to float32 type. If it is `false`, the post-processing layer keeps the tensor type of the application.

For more details about the post-processing parameters, see the *Example: Postprocessing Setups* section in *Vivante Programming: ACUITY Toolkit User Guide*.

2. Modify the export script by adding the `--postprocess` argument, as shown in Line 9 below:

```

1. cmd="$PEGASUS export ovxlib \
2.   --model          ${NAME}.json \
3.   --model-data     ${NAME}.data \
4.   --dtype          ${TYPE} \
5.   --batch-size     1 \

```



```

6.      --save-fused-graph      \
7.      --target-ide-project    'linux64'\
8.      --with-input-meta      ${NAME}_inputmeta.yml \
9.      --postprocess-file      ${NAME}_postprocess_file.yml \
10.     --output-path           ${generate_path}/${NAME}_non-quantized"

```

For more details about the export script, see [Section 2.3.5, Exporting a Network Model to an OpenVX application](#).

3. Run this export script.

This generates an OpenVX application with the post-processing layer added.

Results

Settings in the postprocess section in the post-processing file are ported into the `vnn_post_process.c` file, which is generated together with the OpenVX application.

What to Do Next

Use the following sequence to run the OpenVX application either on the VivanteIDE or on a device with Vivante VIPs:

1. Set the following environment variable:

```
export VSI_USE_IMAGE_PROCESS=1
```

2. Run the application either on the VivanteIDE or on the device.

Note: Post-processing is executed on Vivante PPU and TPUs. In the VivanteIDE, the PPU and TPUs are emulated.

2.4.9 Pre-processing Images for the Unified Driver Use

In the scenario where a network requires a fixed input image size, to feed the network with input images of another size, you need to modify the image size settings in the `vnn_pre_process.c` file exported by ACUTY. This file defines the pre-processing parameter configurations of the network and is generated by the export script.

This section takes the `mobilenet_ssd` network that requires an input image size of $300 \times 300 \times 3$ as an example to describe how to resize input images with a size of $432 \times 432 \times 3$ to the required size.

Prerequisites

Make sure:

- The application for the network is generated with the pre-processing layer added.
For details about the generation, see [Section 2.4.7, Exporting Applications with Pre-Processing Layers Added](#).
- The following environment variable is added before this network is run:

```
export VSI_USE_IMAGE_PROCESS=1
```

Procedure

- From the directory where the network application is generated, open the `vnn_pre_process.c` file.

2. Edit the file to resize the entire input image, as shown from Line 4 to Line 6 in the following example:

```

1. /*pre process for lid: data_0*/
2. vsi_nn_preprocess_source_layout_e source_layout_for_norm_tensor_0 = VSI_NN_SOURCE_LAYOUT_NCHW;
3. vsi_nn_preprocess_source_format_e source_format_for_norm_tensor_0 = VSI_NN_SOURCE_FORMAT_IMAGE_RGB;
4. //vsi_nn_preprocess_image_size_t size_for_norm_tensor_0 = {300, 300, 3};
5. vsi_nn_preprocess_image_size_t size_for_norm_tensor_0 = {432, 432, 3};
6. vsi_nn_preprocess_image_resize_t resize_for_norm_tensor_0 = {300, 300, 3};
7. int8_t reverse_channel_for_norm_tensor_0 = 1;
8. float mean_and_scale_0[] = {127.5, 127.5, 127.5};

```

Where:

- Line 5: The `size_for_norm_tensor_0` structure defines the original size of the input image.
In the example, the original input image size is (432, 432, 3).
- Line 6: The `resize_for_norm_tensor_0` structure defines the new size to scale the input image.
In the example, the new size is (300, 300, 3), as required by the network.

3. Edit the file to resize a rectangle area cropped from the input image, as shown in Lines 6 to 8 and Line 23 in the following example:

```

1. /*pre process for lid: data_0*/
2. vsi_nn_preprocess_source_layout_e source_layout_for_norm_tensor_0 = VSI_NN_SOURCE_LAYOUT_NCHW;
3. vsi_nn_preprocess_source_format_e source_format_for_norm_tensor_0 = VSI_NN_SOURCE_FORMAT_IMAGE_RGB;
4. vsi_nn_preprocess_image_size_t size_for_norm_tensor_0 = {432, 432, 3};
5. vsi_nn_preprocess_image_resize_t resize_for_norm_tensor_0 = {300, 300, 3};
6. int32_t size[3] = {432, 432, 3};
7. int32_t begin[3] = {0, 0, 3};
8. vsi_nn_preprocess_crop_t crop_for_norm_tensor_0 = {begin, size, 3};
9. int8_t reverse_channel_for_norm_tensor_0 = 1;
10. float mean_and_scale_0[] = {127.5, 127.5, 127.5};
11. vsi_nn_preprocess_mean_and_scale_t mean_and_scale_for_norm_tensor_0 = {mean_and_scale_0, 3, 0.007843137};
12. int32_t perm_0[] = {0, 1, 2, 3};
13. vsi_nn_preprocess_permute_t permute_for_norm_tensor_0 = {perm_0, 4};
14. vsi_nn_preprocess_base_t pre_process_for_norm_tensor_0[] =
15. {
16.     {VSI_NN_PREPROCESS_SOURCE_LAYOUT, &source_layout_for_norm_tensor_0},
17.     {VSI_NN_PREPROCESS_SET_SOURCE_FORMAT, &source_format_for_norm_tensor_0},
18.     {VSI_NN_PREPROCESS_IMAGE_SIZE, &size_for_norm_tensor_0},
19.     {VSI_NN_PREPROCESS_IMAGE_RESIZE_BILINEAR, &resize_for_norm_tensor_0},
20.     {VSI_NN_PREPROCESS_REVERSE_CHANNEL, &reverse_channel_for_norm_tensor_0},
21.     {VSI_NN_PREPROCESS_MEAN_AND_SCALE, &mean_and_scale_for_norm_tensor_0},
22.     {VSI_NN_PREPROCESS_PERMUTE, &permute_for_norm_tensor_0},
23.     {VSI_NN_PREPROCESS_CROP, &crop_for_norm_tensor_0},
24. };
25. /*{graph_input_idx, preprocess}*/
26. const static vsi_nn_preprocess_map_element_t preprocess_map[] =
27. {
28.     {0, pre_process_for_norm_tensor_0, sizeof(pre_process_for_norm_tensor_0) /
29.     sizeof(vsi_nn_preprocess_base_t)},

```

Where:

- Line 6 to Line 8: The `crop_for_norm_tensor_0` structure defines the rectangle area in the input image to be cropped and scaled to the new size.
In the example, the full image is resized.
- Line 14: The `pre_process_for_norm_tensor_0[ ]` structure defines pre-processing parameters.
- Make sure:
 - Line 18 and Line 19 are present.
 - Line 23 is added manually.

Results

When integrating the network application with the unified driver, images in the dataset are automatically scaled to (300, 300, 3), as required by the network.

3 VivantelIDE

This chapter describes how to run applications in VivantelIDE for simulation.

VivantelIDE provides a command line tool and a graphics user interface (GUI) for you to run, debug, and profile OpenVX projects. For more information about VivantelIDE, see *VivantelIDE User Guide*.

3.1 Using VivantelIDE GUI

This section describes how to run, debug, and profile OpenVX projects by using the VivantelIDE GUI.

3.1.1 VivantelIDE Packages

VivantelIDE packages include:

- Vivante_IDE-{version}-Linux-x86_{sub-version}
- Vivante_IDE-{version}-Win-x32_{sub-version}

Important: When launching VivantelIDE, always use the shortcut or the `vivanteide5.7.1.bat` script for windows and the `vivanteide5.7.1` script for Linux.

3.1.2 VivantelIDE License

Before installing VivantelIDE, apply a license from the following website:

<https://www.verisilicon.com/cn/VIPAcuityIDELicenseRequest>

Important: Do not install VivantelIDE on virtual machines where the license is invalid.

3.1.3 Importing an OpenVX Project

Before configuring and running an OpenVX project for simulation in VivantelIDE, you need to import it into VivantelIDE.

Prerequisites

Make sure that an OpenVX project is available and located in a different directory from the workspace. For information about how to export an OpenVX project by using ACUITY Toolkit, see [Section 2.3.5, Exporting a Network Model to an OpenVX application](#).

Procedure

1. From the main menu, select **File > Import**.
2. In the displayed **Import - Select** dialog, choose **General > Existing Projects into Workspace** and click **Next**.
3. Click **Browse** on the right of the **Select root directory** text box and select the directory of the unified network project.
4. In the **Projects** list box, select the unified network project.
5. Select the **Copy projects into workspace** check box and click **Finish**.

Result

The project is imported and viewable in the **Project Explorer** pane.

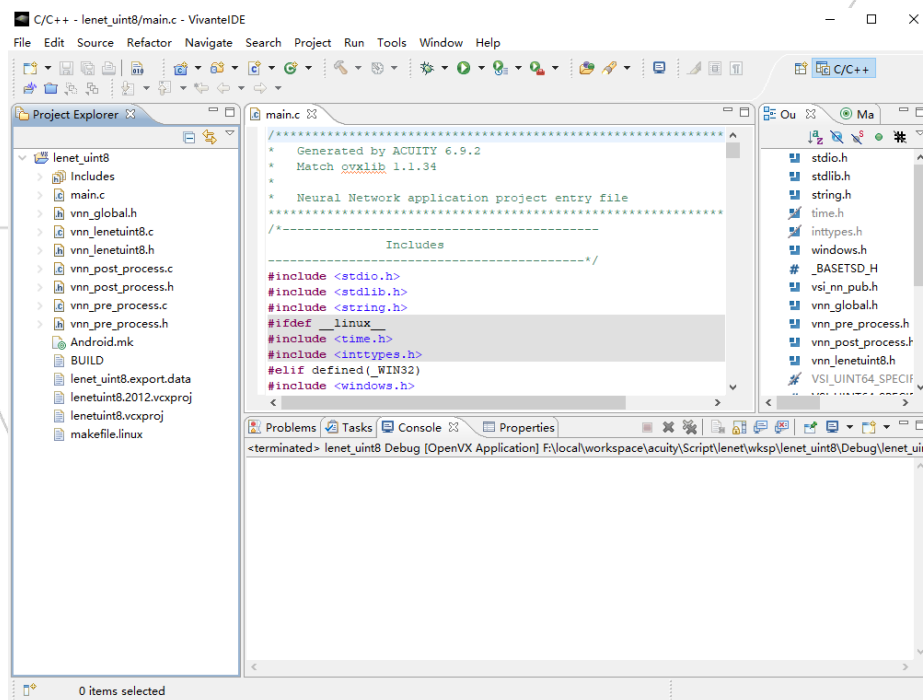


Figure 3. OpenVX Project Imported

3.1.4 Running an OpenVX Project

To run an OpenVX project, you need to build the project and configure the runner. After the execution is completed, verify that the execution result is correct.

Prerequisite

Make sure:

- The OpenVX project is imported into VivanteIDE, as described in [Section 3.1.3, Importing an OpenVX Project](#).
- (Optional) The network of the OpenVX project is profiled, as described in [Section 3.1.5, Profiling a Network](#).

Procedure

1. In the **Project Explorer** pane of VivanteIDE, right-click the project, and then click **Build Project**.

Note: You do not need to modify the build settings.

- If the build succeeds, a debug folder with an executable program `<project name>.exe` is generated and displayed under the current project in the **Project Explorer** pane.
- If the build fails, no debug folder is generated.

Check the settings for the ACUITY OpenVX generation and the settings in the import procedure in VivanteIDE, and then re-import the project.

2. Right-click the project, point to **Run As**, and then click **Run Configurations**.

The **Run Configurations** dialog appears.

- The left pane shows the list of applications available for execution.
- The right pane contains the configurations of the application execution, including the **Main**, **Arguments**, **Runner**, **Environment**, and **Common** tabs.

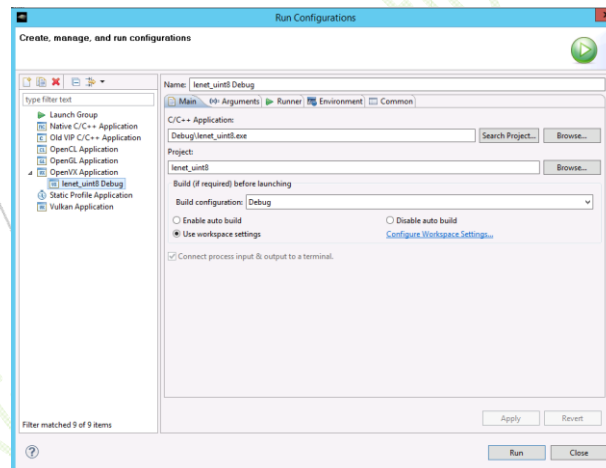


Figure 4. Run Configurations Dialog

3. In the left pane, double-click **OpenVX Application**.

The `<project name> Debug` application is generated and displayed under **OpenVX Application**.

4. Click the generated debug application, and then set the parameters in the right pane.

- On the **Arguments** tab, enter the file paths of the network model file and the input data.

The model file is the `<project name>.export.data` file exported from ACUITY. The input data is one or more `.jpg` or `.tensor` files. Separate the file paths with spaces.

Type multiple input files only for multiple-input networks. The file order must follow the input order of the network.

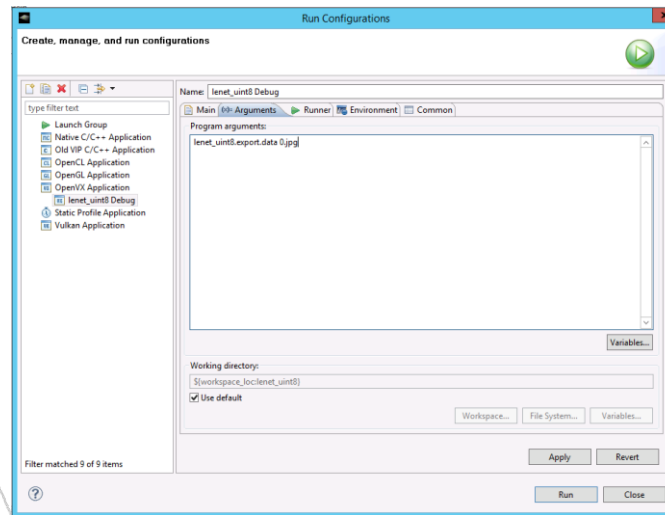


Figure 5. Arguments Tab of the Run Configurations Dialog

- On the **Runner** tab, select the hardware target.

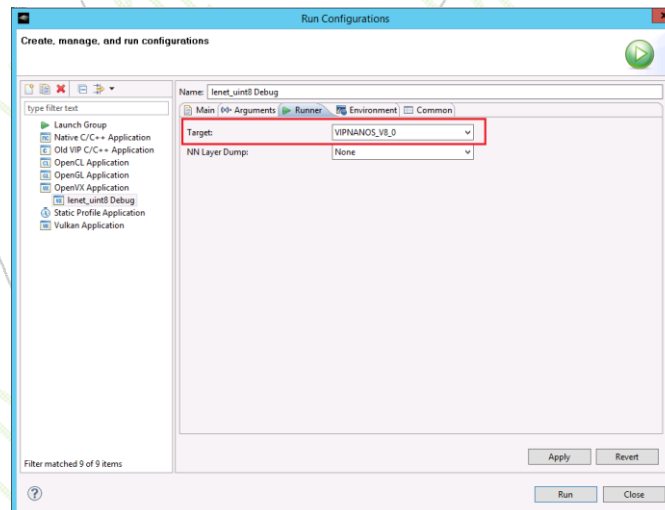


Figure 6. Runner Tab of the Run Configurations Dialog

- On the **Environment** tab, set environment variables.
 - To save the NBG file, set the VIV_VX_ENABLE_SAVE_NETWORK_BINARY variable to 1.
After execution, the NBG file `network_binary.nb` is saved in the working directory.
For information about how to use the NBG file, see [Section 6.3, Configuring Network Profiling for VIPLite Driver](#) and [Section 4.6.1.3, Creating NBG Applications](#).
 - To dump the input and output tensors of each layer, set the NN_LAYER_DUMP variable to 1.
When the output tensor obtained through VivanteIDE is different from that obtained through ACUITY, you can compare the dump results from VivanteIDE with those from ACUITY to locate erroneous layers. The dump results are saved in a folder named LayerDump. For details, see [Section 3.1.6, Comparing Dump Results](#).
For more available environment variables, see [Section 5, Environment Variables](#).
- On the **Main** and **Common** tabs, do not modify the settings.

5. Click Run.

Results

- In the **Console** view of the **C/C++** perspective, the network structure and execution results are printed.

```

D [print_tensor:136]in(1) : id[ 8] vtl[0] const[1] shape[ 500, 10] } fnc[us ] qnt[ASM sp=135, scale=0.001580]
D [print_tensor:136]in(2) : id[ 9] vtl[0] const[1] shape[ 10] } fnc[132] qnt[ASM sp= 0, scale=0.000032]
D [print_tensor:136]out(0) : id[ 14] vtl[1] const[0] shape[ 10, 1] } fnc[us ] qnt[ASM sp= 80, scale=0.042750]
D [setup_node:432]Setup node id[7] uid[8] op[SOFTMAX]
D [print_tensor:136]in(0) : id[ 14] vtl[1] const[0] shape[ 10, 1] } fnc[us ] qnt[ASM sp= 80, scale=0.042750]
D [print_tensor:136]out(0) : id[ 1] vtl[0] const[0] shape[ 10, 1] } fnc[116] qnt[NONE]
D [optimize_node:576]Backward optimize neural network
D [optimize_node:583]Forward optimize neural network
D [vsi_nn_internal_optimize_node:614]Optimize node uid[8] sub_uid[1] op[SOFTMAX_INTERNAL]
I [compute_node:325]Create vx node
D [vsi_nn_internal_compute_node:587]Compute node uid[8] sub_uid[1] op[SOFTMAX_INTERNAL]
D [_check_swapped_tensors:91]Check swapped tensors
Create Neural Network: 0ms or 94us
Verify...
Verify Graph: 44ms or 44520us
Start run graph [1] times...
Run the 1 time: 1.00ms or 1893.00us
vxSuccess@Graph execution time:
Total 1.00ms or 1893.00us
Average 1.85ms or 1893.00us

--- Tops ---
0: 0.999512
5: 0.000144
6: 0.000144
2: 0.000099
1: 0.000000
  
```

Figure 7. OpenVX Project Execution Results

- In the project directory, a network input `.txt` file and a network output `.txt` file are generated.
If the NN_LAYER_DUMP variable is set to 1 during configuration, the input and output tensors of each layer are also generated in the project directory.
Note: Make sure that the data in these files matches the input and output data generated in ACUITY.

3.1.5 Profiling a Network

To profile a network with the unified driver in VivanteIDE, you need to configure the profiler. With the returned network performance data, you can estimate the FPS and bandwidth consumed on VIP cores.

Prerequisites

Make sure that the execution result of the OpenVX project is correct. For details, see [Section 3.1.4, Running an OpenVX Project](#).

Procedure

1. In the **Project Explorer** pane of VivanteIDE, right-click the project, point to **Profile As**, and then click **Profile Configurations**.

The **Profile Configurations** dialog appears.

- The left pane shows the list of applications available for profiling.
- The right pane contains the configurations of the profiling, including the **Main**, **Environment**, **Profiler**, and **Source** tabs.

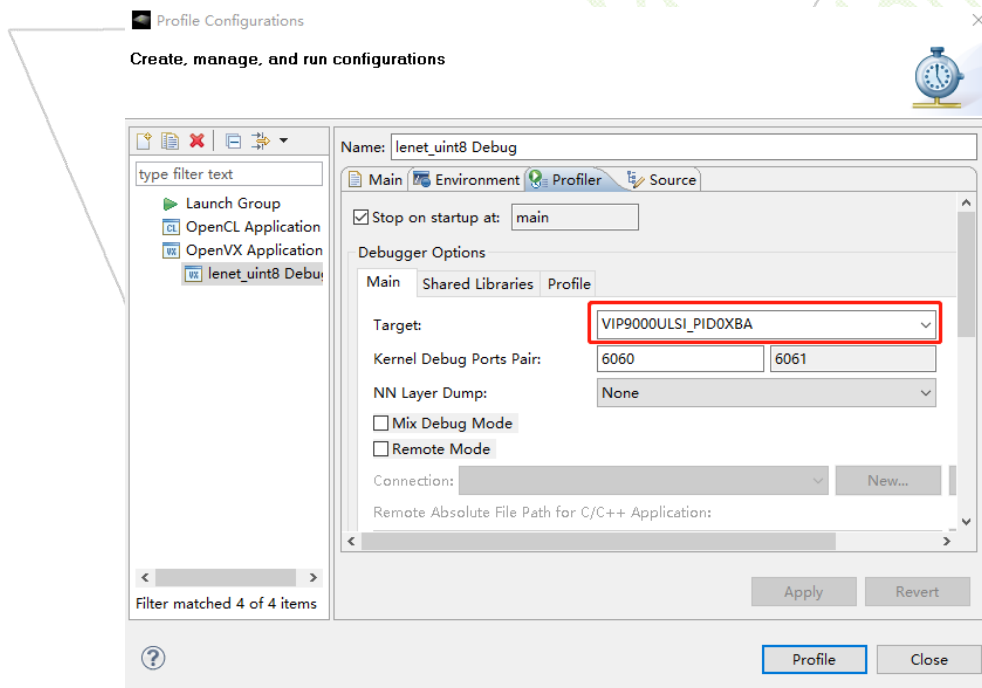


Figure 8. Profile Configurations Dialog

2. Click the debug application under **OpenVX Application** in the left pane, and then set the parameters in the right pane.

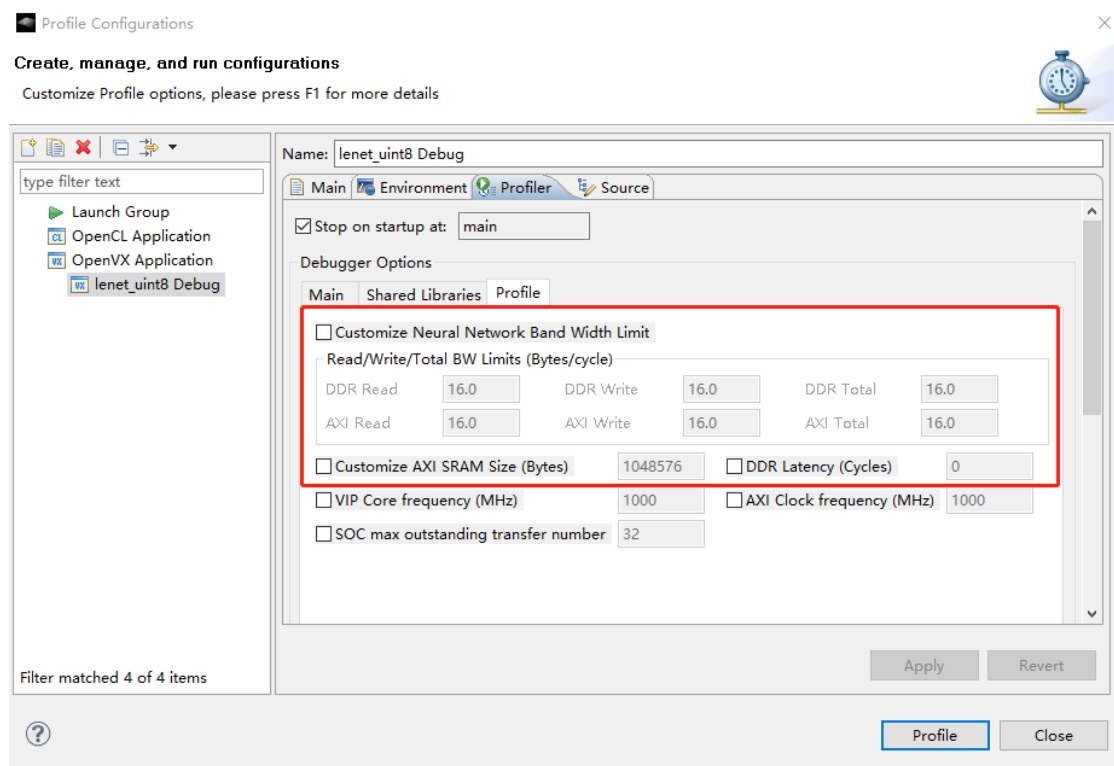


Figure 9. Profile Sub-Tab of the Profiler Tab

- On the **Profiler** tab, complete the settings on the three sub-tabs:
 - On the **Main** sub-tab, set the parameter listed in the following table.

Table 8. Parameter on the Main Sub-tab

Parameter	Description	Configuration Notes
Kernel Debug Ports Pair	The vSimulator port and the VDT proxy port	The default port numbers are 6060 and 6061. If a port is occupied, use an idle port instead.

- On the **Profiler** sub-tab, set the parameters listed in the following table to match the actual hardware environment. Do not modify the settings of other parameters.

Note: The more accurate the parameter settings are, the more accurate the generated network performance data is.

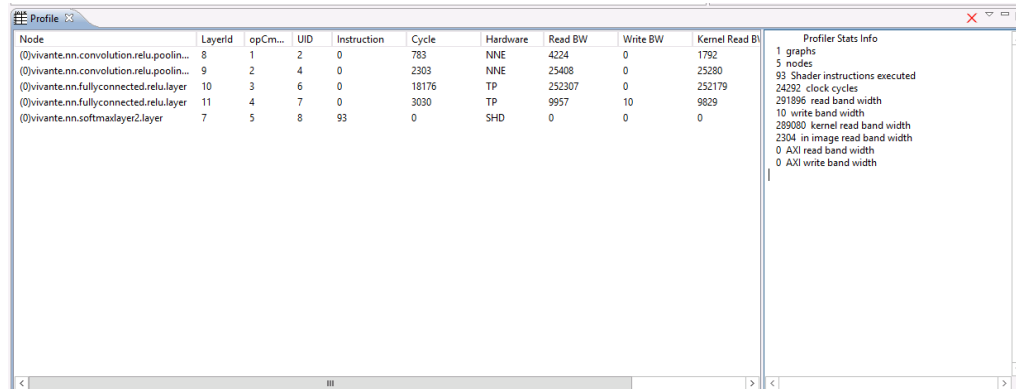
Table 9. Parameters on the Profiler Sub-tab

Parameter	Description	Configuration Notes
• DDR Read • DDR Write • DDR Total	The maximum number of data bytes that can be read or/and write to the DDR SRAM each VIP cycle	N/A
• AXI Read • AXI Write • AXI Total	The maximum number of data bytes that can be read or/and write to the AXI SRAM each VIP cycle	Set these parameters if the Customize AXI SRAM Size check box is selected.
Customize AXI SRAM Size	The size of the AXI SRAM, in bytes	Select the check box only if the actual hardware configurations include the AXI SRAM. If the check box is selected, enter the size of the AXI SRAM in the text box and set the AXI Read , AXI Write , and AXI Total parameters.
DDR Latency	The DDR latency, in cycles	N/A

- On the **Shared Libraries** sub-tab, do not modify the settings.
 - On the **Main**, **Environment**, and **Source** tabs, do not modify the parameter settings.
 - The settings are inherited from the **Run Configurations** dialog in [Section 3.1.4, Running an OpenVX Project](#).
- Click **Profile**.
VivanteIDE starts loading the program and switches to the **Profile** perspective. After the loading is completed, the program pauses.
 - Click **Resume** or enter F8 to resume the program.
The profiling starts, which may take several minutes to finish if the network size is large.

Results

After the profiling is completed, the results are displayed in the **Profile** view. For detailed description, see the [Profile View in VivanteIDE](#) section.



Node	LayerId	opCm...	UID	Instruction	Cycle	Hardware	Read BW	Write BW	Kernel Read B
(0)vivante.nn.convolution.relu.poolin...	8	1	2	0	783	NNE	4224	0	1792
(0)vivante.nn.convolution.relu.poolin...	9	2	4	0	2303	NNE	25408	0	25280
(0)vivante.nn.fullyconnected.relu.layer	10	3	6	0	18176	TP	252307	0	252179
(0)vivante.nn.fullyconnected.relu.layer	11	4	7	0	3030	TP	9957	10	9829
(0)vivante.nn.softmaxlayer2.layer	7	5	8	93	0	SHD	0	0	0

Profiler Stats Info

- 1 graphs
- 5 nodes
- 93 Shader instructions executed
- 24292 clock cycles
- 291896 read band width
- 10 write band width
- 250080 kernel read band width
- 2304 in image read band width
- 0 AXI read band width
- 0 AXI write band width

With the values in the **Profile Stats Info** pane of the **Profile** view, calculate the FPS and DDR total bandwidth as follows:

- FPS: Use the **shader instructions executed** and **clock cycles** values.

$$\text{FPS} = F / (\text{Shader instructions executed} / \text{PPU_NUM} + \text{Clock cycles})$$

where:

- PPU_NUM: Indicates the number of PPU's in the VIP core.
- Shader instructions executed**: One cycle executes one instruction.
- F: The clock frequency.

- DDR total bandwidth, in megabytes per inference: Use the **read band width** and **write band width** values.

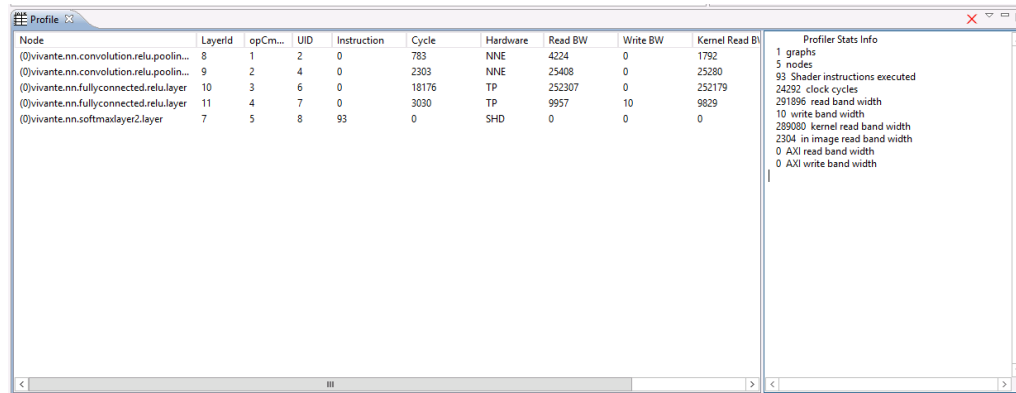
Note: The DDR total bandwidth per inference obtained here includes only the bandwidth consumed on the TP engine and NN engine. The DDR read bandwidth and write bandwidth of shader nodes in VivanteIDE are invalid.

$$\begin{aligned} \text{DDR total bandwidth per inference (MBytes)} \\ = (\text{Read band width} + \text{Write band width}) / (1024 \times 1024) \end{aligned}$$

For details about the values in the **Profile** view, see the [Profile View in VivanteIDE](#) section.

3.1.5.1 Profile View in VivanteIDE

After the profiling is completed, the **Profile** view appears and displays two panes.



Node	LayerId	opCmd...	UID	Instruction	Cycle	Hardware	Read BW	Write BW	Kernel Read B
(0)vivante.nn.convolution.relu.poolin...	8	1	2	0	783	NNE	4224	0	1792
(0)vivante.nn.convolution.relu.poolin...	9	2	4	0	2303	NNE	25408	0	25280
(0)vivante.nn.fullyconnected.relu.layer	10	3	6	0	18176	TP	252307	0	252179
(0)vivante.nn.fullyconnected.relu.layer	11	4	7	0	3030	TP	9957	10	9829
(0)vivante.nn.softmaxlayer2.layer	7	5	8	93	0	SHD	0	0	0

Profiler Stats Info

```

1 graphs
5 nodes
93 Shader instructions executed
24292 clock cycles
291896 read band width
10 write band width
289080 kernel read band width
2304 in image read band width
0 AXI read band width
0 AXI write band width

```

- Left pane: Lists the profile information of each node.
 - Node:** The name of the node in the graph.
 - LayerId:** Omit this column because it is for internal use only.
 - opCmdId:** The sequence to execute the node.
 - UID:** The node ID assigned by the unified driver as follows:
 - If a node corresponds to one ACUITY layer, the node takes the layer ID.
 - If a node corresponds to multiple ACUITY layers, the node takes the ID of the last layer.
 - If multiple nodes correspond to the same ACUITY layer, the last node share the same ID as the layer. The IDs of the other nodes are given -1.
 You can query the UID in the following files:
 - The network model file in ACUITY, the .json file
The UID is indicated by the suffix of each layer name `XXX_<UID>`.
 - The `vnn_<project name>.c` file exported from ACUITY
The UID is indicated by the last parameter of `NEW_VXNODE(...,UID)` in the `vnn_Create<project name>()` function.
 - Instruction:** The number of instructions required by the node on the PPU engine. One cycle executes one instruction.
 - Cycle:** The number of VIP cycles consumed on the TP engine and NN engine to execute the node.
 - Hardware:** The VIP computing engine, the PPU, TP, or NN engine, that executes the node. In VIP, PPU refers to the shader core.
 - Read BW and Write BW:** The DDR read and write bandwidth of each node, in bytes per inference.
Note: The DDR read bandwidth and write bandwidth of shader nodes in VivanteIDE are invalid.
 - Kernel Read BW:** The DDR read bandwidth consumed by weight/bias data.
 - In Image Read BW:** The DDR read/write bandwidth consumed by input and output data, including intermediate data.

- Right pane (**Profile Stats Info**): Summarizes the network performance.
 - **Graphs**: The number of graphs executed.
 - **Nodes**: The number of nodes in the graphs.
 - **Shader instructions executed**: The number of instructions required on the PPU engine. One cycle executes one instruction.
 - **Clock cycles**: The number of VIP cycles consumed on the TP engine and NN engine.
 - **Read band width**: The DDR read bandwidth, in bytes per inference.
 - **Write band width**: The DDR write bandwidth, in bytes per inference.
 - **Kernel Read BW**: The DDR read bandwidth consumed by weight/bias data.
 - **In Image Read BW**: The DDR read/write bandwidth consumed by input and output data, including intermediate data.
 - **AXI read band width**: The AXI read bandwidth, in bytes per inference.
 - **AXI write band width**: The AXI write bandwidth, in bytes per inference.

For information about how to use these values to calculate the FPS and the DDR total bandwidth, see [Section 3.1.5, Profiling a Network](#).

3.1.6 Comparing Dump Results

You can compare the dump results from ACUITY, those from VivanteIDE, and those from drivers to locate erroneous layers.

Prerequisites

Make sure that two dump result files are available. For information about how to dump the input and output tensors of a network, see the following sections:

- [Section 3.1.4, Running an OpenVX Project](#) for the method of dumping network tensors from VivanteIDE
- [Section 2.3.6, Dumping Tensors](#) for the method of dumping network tensors from ACUITY
- [Section 5.1.3, Environment Variables for OpenVX Unified Driver Configuration](#) for the method of dumping network tensors from an FPGA with the unified driver

Procedure

1. In the **Project Explorer** pane, double-click the .json file of the network model.

The diagram of network model is displayed.

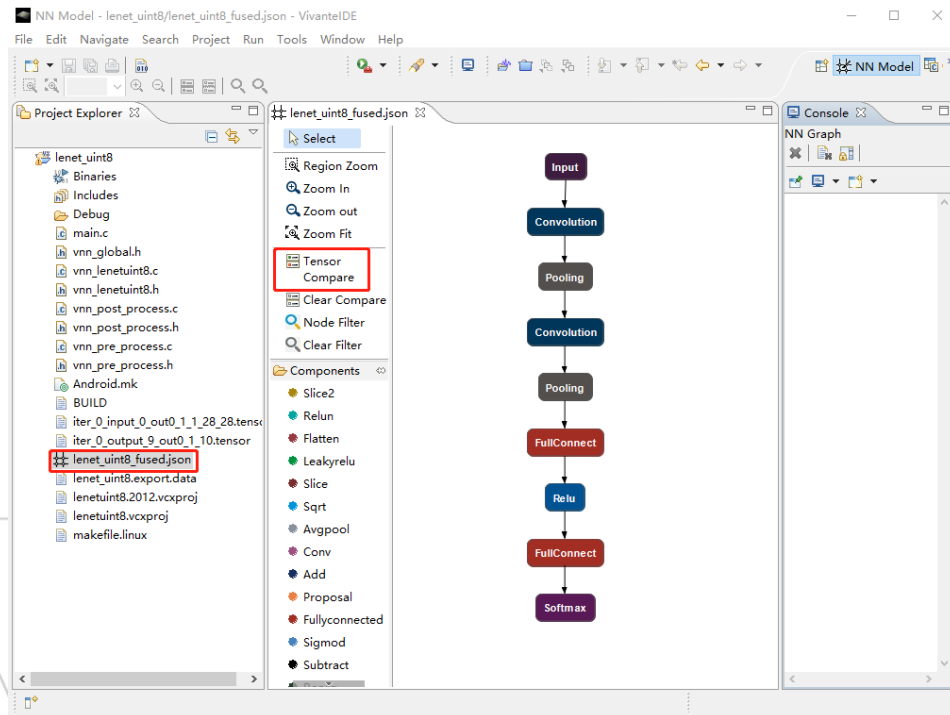


Figure 10. Tensor Compare Function Entry

2. Click **Tensor Compare** in the palette.
3. In the displayed **Tensor Compare** dialog, configure the comparison.

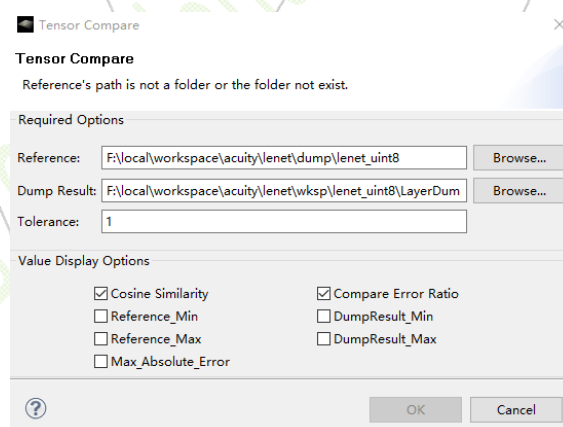


Figure 11. Tensor Compare Dialog

- Click **Browse** next to the **Reference** text box and select the golden dump result file.
- Click **Browse** next to the **Dump Results** text box and select the dump result file to be compared.
- Set **Tolerance** to a value, which is used to calculate the error rate tolerance.
- Select the **Cosine Similarity** check box if you need to show the cosine similarity value between the reference and dump result in node content.
- Select the **Compare Error Ratio** check box if you need to show the error ratio between the reference and dump result in node content.
- Select the **Reference_Min** check box if you need to show the minimum value of the reference in node content.
- Select the **Reference_Max** check box if you need to show the maximum value of the reference in node content.
- Select the **DumpResult_Min** check box if you need to show the minimum value of the dump result in node content.
- Select the **DumpResult_Max** check box if you need to show the maximum value of the dump result in node content.
- Select the **Max_Absolute_Error** check box if you need to show the maximum absolute error value between the reference and dump result in node content.

Result

The diagram of the network model is displayed. For each matched NN node, the diagram shows a box that contains values for the selected check boxes.

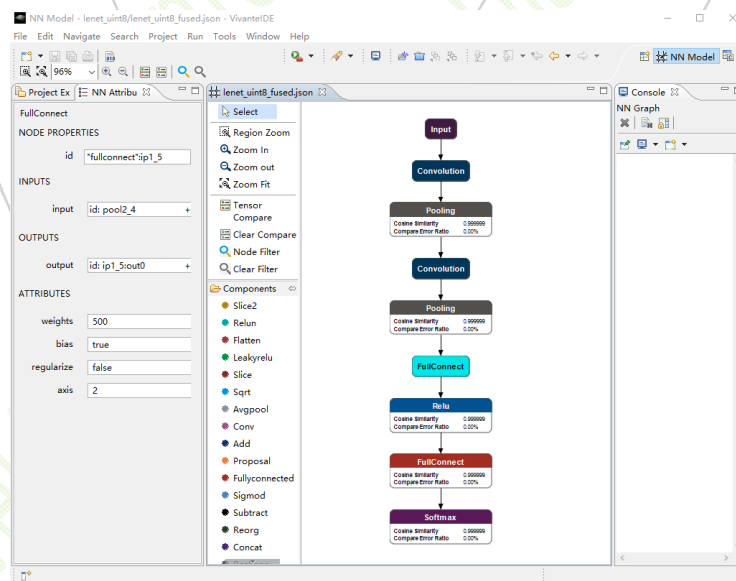


Figure 12. Dump Result Comparison Output

Note 1: To ensure that the input for ACUITY and that for VivanteIDE are consistent, use the .tensor input format. This is because .jpg file may use different JPG decoding libraries and the decoded data differs, which makes error location more difficult.

Note 2: The network may be optimized by the driver and some layers may be merged. If a layer does not exist in the VivanteIDE or FPGA dump file, it may be merged into another layer. Skip the missing layer during comparison.

Note 3: A difference less than the fl (fixed position length) or scale value between the dump result from ACUITY and that from VivanteIDE is acceptable. However, there are exceptions. For example, the RPN of a Faster R-CNN network has a difference in the position of the calculation box, which causes the data after ROI Pooling to be inconsistent. In this case, you can check the results through post-processing to judge the correctness.

3.2 Using VivanteIDE Command Line Tool

This section describes how to run, debug, and profile OpenVX projects by using the VivanteIDE command line tool.

3.2.1 Installing a vSimulator SDK Package

Before using the Vivante command line tool, you need to have it installed with a vSimulator SDK package.

Prerequisites

Make sure that a vSimulator SDK package Vivante_simulator_SDK_<version>.tgz with version 6.4.14 or later is available.

Procedure

1. Extract the vSimulator SDK package by running the following command:

```
tar xzvf Vivante_simulator_SDK_<version>.tgz
```

2. Set environment variables for building and running as follows:

```
export AQR00T=`pwd`
export LD_LIBRARY_PATH=$AQR00T/x86_64_linux/lib/
export USE_IDE_LIB=1
export VIVANTE_SDK_DIR=$AQR00T/x86_64_linux
```

3.2.2 Building an OpenVX Project

Before configuring and running an OpenVX project for simulation in VivanteIDE, you need to build it in VivanteIDE.

Prerequisites

Make sure:

- A vSimulator SDK package is installed, as described in [Section 3.2.1, Installing a vSimulator SDK Package](#).
- An OpenVX application is available. For information about how to export an OpenVX application by using ACUITY Toolkit, see [Section 2.3.5, Exporting a Network Model to an OpenVX application](#).

Procedure

1. Go to the directory of the OpenVX application as follows:

```
cd OVX_DIR
```

2. Build the OpenVX project with the following command:

```
make -f makefile.linux
```

3.2.3 Running an OpenVX Project

To run an OpenVX project, you need to build the project and configure the runner. After the execution is completed, verify that the execution result is correct.

Prerequisite

Make sure:

- The OpenVX project is built in VivanteIDE, as described in [Section 3.2.2, Building an OpenVX Project](#).
- (Optional) The network of the OpenVX project is profiled, as described in [Section 3.2.4, Profiling a Network](#).

Procedure

1. Set the hardware target to the vSimulator configuration name `VIP9000*_PID0X*`, for example:

```
export VSIMULATOR_CONFIG=VIP9000ULSI_PID0XBA
```

Note: To obtain the vSimulator configuration name, contact VeriSilicon support.

2. (Optional) Enable generating an NBG from the OpenVX project after execution as follows:

```
export VIV_VX_ENABLE_SAVE_NETWORK_BINARY=1
```

For more information about environment variables, see [Chapter 5, Environment Variables](#).

3. Run the OpenVX project by running the following command (lenetuint8 as an example):

```
./lenetuint8 lenet_uint8.export.data iter_0_input_0_out0_1_1_28_28.tensor
```

Results

The following script shows an example of the runtime information returned when an OpenVX project is run:

```
Create Neural Network: 3ms or 3218us
Verify...
Verify Graph: 1642ms or 1642121us
Start run graph [1] times...
Run the 1 time: 255.47ms or 255467.55us
vxProcessGraph execution time:
Total 255.48ms or 255480.89us
Average 255.48ms or 255480.89us
--- Top5 ---
0: 1.000000
1: 0.000000
2: 0.000000
3: 0.000000
4: 0.000000
```

3.2.4 Profiling a Network

To profile a network with the unified driver in VivanteIDE, you need to configure the profiler. With the returned network performance data, you can estimate the FPS and bandwidth consumed on VIP cores.

Prerequisites

Make sure that the execution result of the OpenVX project is correct. For details, see [Section 3.2.3, Running an OpenVX Project](#).

Procedure

1. Configure the profiler by setting the following environment variables:

```
export VSIMULATOR_DEBUG=4
export DISABLE_IDE_DEBUG=1
export VIV_VX_COLLECT_PERF_TYPE=1
```

2. (Optional) Control the bandwidth and SRAM size through the following environment variables:

```
export NN_EXT_DDR_READ_BW_LIMIT=32 // Total DDR read BW, in bytes per cycle
export NN_EXT_DDR_WRITE_BW_LIMIT=32 // Total DDR write BW, in bytes per cycle
export NN_EXT_DDR_TOTAL_BW_LIMIT=32 // Total DDR total BW, in bytes per cycle
export NN_EXT_AXI_SRAM_READ_BW_LIMIT=32 // Total AXISRAM read BW, in bytes per cycle
export NN_EXT_AXI_SRAM_WRITE_BW_LIMIT=32 // Total AXISRAM read BW, in bytes per cycle
export NN_EXT_AXI_SRAM_TOTAL_BW_LIMIT=32 // Total AXISRAM read BW, in bytes per cycle
export NN_EXT_VIP_SRAM_SIZE=4194304 // VIP SRAM (L1) size per cluster, in bytes
export NN_EXT_AXI_SRAM_SIZE=4194304 // Total AXI SRAM (L2) size, in bytes
export VIV_VX_DDR_LATENCY=200 // DDR/AXI SRAM latency, in cycles
```

3. Profile the network by running the following command (lenetuint8 as an example):

```
./lenetuint8 lenet_uint8.export.data iter_0_input_0_out0_1_1_28_28.tensor
```

Results

The following script shows an example of the information returned when a network is profiled. It consists of runtime information, per-layer profiles, and a summary.

Note 1: During network profiling, all the network outputs are 0 because the profiler skips model computing.

Note 2: The PPU cycle count comes from PPU instruction estimation. Therefore, there may be a difference between network profiling and hardware execution results.

```
Create Neural Network: 13ms or 13651us
Verify...
Verify Graph: 1598ms or 1598313us
Start run graph [1] times...

LayerID operationID opCmdID UID INSTRUCTION CYCLE HADRWARE READBW WRITEBW kernelReadBW inImageReadBW MAC
MACUtil NODE
9 0 1 2 0 1881 NNE 2112 0 832 1152 0 0.00% vivante.nn.convolution.relu.pooling.layer2
10 0 2 4 0 7571 NNE 25024 0 24896 0 0 0.00% vivante.nn.convolution.relu.pooling.layer2
12 0 3 6 0 109363 NNE 412288 0 412160 0 0 0.00% vivante.nn.convolution.relu.pooling.layer2
11 0 4 7 0 14681 TP 19200 0 16457 0 0 0.00% vivante.nn.fullyconnected.relu.layer
7 0 5 8 0 122 SHD 0 20 0 0 0 0.00% vivante.nn.softmaxlayer2.layer
```

Profiler Stats Info

1 graphs

5 nodes

0 instructions executed

133618 NN clock cycles

458624 NN read band width

20 NN write band width

0 NN AXI SRAM read band width

0 NN AXI SRAM write band width

0 MAC



4 Unified Driver

This chapter describes how to run networks and integrate networks into applications with Vivante GALVIP unified driver.

For more information about the unified driver, see the reference documents listed in the [References](#) section.

4.1 Unified Driver Packages

Vivante GALVIP unified driver packages include:

- Vivante_GALVIP_Unified_Src_drv_{version}.tgz
- Vivante_GALVIP_Unified_Src_drv_{customer}_GCFeature-addon_{version}.org.tgz
- Vivante_GALVIP_Unified_Src_drv_OCL-addon_{version}.noGPL.tgz
- Vivante_GALVIP_Unified_Src_drv_OVX-addon_{version}.noGPL.tgz
- Vivante_GALVIP_Unified_Src_drv_OVXLIB-addon_{version}.tgz
- Vivante_GALVIP_Unified_Src_drv_SPIRV-addon_{version}.noGPL.tgz
- Vivante_GALVIP_Unified_Src_drv_{gpu_config}_VXC-addon_{version}.tgz
- Vivante_GALVIP_Unified_Src_drv_{platform}-prebuilt-addon_{version}.tgz
- Vivante_GALVIP_Unified_Src_kernel_{version}.tgz

There may be other additional libraries or patches when needed.

4.2 Building the Unified Driver

4.2.1 Building the Driver

Before Vivante GALVIP unified driver can be loaded, you need to build it.

Procedure

1. Extract all the driver packages to the same directory as follows:

```
tar xzvf Vivante_GALVIP_Unified_Src_drv_{version}.tgz
tar xzvf Vivante_GALVIP_Unified_Src_drv_{customer}_GCFeature-addon_{version}.org.tgz
tar xzvf Vivante_GALVIP_Unified_Src_drv_OCL-addon_{version}.noGPL.tgz
tar xzvf Vivante_GALVIP_Unified_Src_drv_OVX-addon_{version}.noGPL.tgz
tar xzvf Vivante_GALVIP_Unified_Src_drv_OVXLIB-addon_{version}.tgz
tar xzvf Vivante_GALVIP_Unified_Src_drv_SPIRV-addon_{version}.noGPL.tgz
tar xzvf Vivante_GALVIP_Unified_Src_drv_{gpu_config}_VXC-addon_{version}.tgz
tar xzvf Vivante_GALVIP_Unified_Src_drv_{platform}-prebuilt-addon_{version}.tgz
tar xzvf Vivante_GALVIP_Unified_Src_kernel_{version}.tgz
```

2. (Optional) Apply patches if any.

3. In the `driver_build_sample.sh` file, configure the following options:

Important: If you have the tool chain specified, provide it to [Vivante Technical Support](#) in advance for building libraries.

```
arm)
    export ARCH_TYPE=$ARCH
    export CPU_TYPE=arm920
    export FIXED_ARCH_TYPE=arm
    export KERNEL_DIR=/home/software/Linux/linux-2.6.21-arm1
    export CROSS_COMPILE=arm-none-linux-gnueabi-
    export TOOLCHAIN=/home/software/Linux/toolchain
    export LIB_DIR=$TOOLCHAIN/arm-none-linux-gnueabi/libc/usr/lib
    export USE_VXC_BINARY=1.
    export GPU_CONFIG={gpu_config}
```

Build options:

```
BUILD_OPTION_USE_OPENCL=1
BUILD_OPTION_USE_OPENVX=1
BUILD_OPTION_USE_VIP_ONLY=1
BUILD_OPTION_USE_OVXLIB=1
```

Build options for hardware that supports 40-bit virtual address space:

```
BUILD_OPTION_USE_OPENCL=1
BUILD_OPTION_USE_OPENVX=1
BUILD_OPTION_USE_VIP_ONLY=1
BUILD_OPTION_USE_OVXLIB=1
BUILD_OPTION_VIVANTE_ENABLE_40BITS_VA=1
```

4. Run the following command to build the driver:

```
./driver_build_sample.sh arm
```

5. Save `driver_build_sample.sh` as `cmd.sh`.

6. Add the following line to the end of the `cmd.sh` file:

```
export cc="make -j1 -f makefile.linux $BUILD_OPTIONS"
```

7. Run the following commands to build the driver:

```
source cmd.sh
$cc clean;$cc install
```

If the hardware supports 40-bit virtual address space, run the following commands instead:

```
cd ovxlib
cp vsi_feature_config_64x_40bit_va vsi_feature_config
bash gcc_gen_feature_config_header.sh `pwd`
$cc clean;$cc install
```

8. Build the OVXLIB and other applications.

The following commands take OVXLIB as an example:

```
cd ovxlib
$cc clean;$cc install
```


4.2.2 Porting the Driver

Porting is necessary if it is the first time you build the driver or if errors occur during driver build.

Procedure

1. Create two platform code files from the default platform code as follows:

```
hal/os/linux/kernel/platform/${soc_vendor}/gc_hal_kernel_platform_${(soc_board)}.c
hal/os/linux/kernel/platform/${soc_vendor}/gc_hal_kernel_platform_${(soc_board)}.config
```

where:

- soc_vendor is the first word of SOC_PLATFORM in the .config file.
- soc_board is the last word of SOC_PLATFORM in the .config file.

2. Modify the value of SOC_PLATFORM in the .config file as follows:

```
export SOC_PLATFORM=${(soc_vendor)}-${(soc_board)}
```

3. Modify the gc_hal_kernel_platform_\${(soc_board)}.c file, as instructed in *Vivante Programming: Platform Integration Guide* document.

The following script is an example of the updated file:

Note: The _AdjustParam function in the following example uses default settings if no parameter is set through the insmod command. If a parameter is set using the insmod command, the last command setting takes effect.

```
gceSTATUS _AdjustParam(IN gcsPLATFORM *Platform,OUT gcsMODULE_PARAMETERS *Args)
{
    struct resource *res = NULL;
    struct platform_device *pdev = Platform->device;
    int irqLine = platform_get_irq_byname(pdev, "galcore");

    if (irqLine < 0) {
        irqLine = platform_get_irq(pdev, 0);
        printk("galcore irq number is %d\n", irqLine);
        if (irqLine < 0)
            return gcvSTATUS_OUT_OF_RESOURCES;
    }
    /*NPU interrupt number, The irq should be config at linux dts file, NPU use HIGH_LEVEL signal to
    trigger interrupt*/
    Args->irqs[gcvCORE_MAJOR] = irqLine;

    Args->registerBases[0] = 0xFF100000; /*NPU register base address*/
    Args->registerSizes[0] = 2 << 10;
    Args->contiguousBase = 0xc0000000; /*if the system reserve video memory for NPU, please set physical
    address to contiguousBase, set 0 to contiguousBase , kernel driver will allocate video memory
    dynamically*/
    Args->contiguousSize = 0x400000; /*reserve video memory size*/

    return gcvSTATUS_OK;
}

gceSTATUS _GetPower(IN gcsPLATFORM *Platform)
{
    /*implemenet it to power on NPU*/
    return gcvSTATUS_OK;
}
```

```

gceSTATUS _SetPower(IN gcsPLATFORM * Platform,IN gceCORE GPU,IN gctBOOL Enable)
{
    /*implemenet it to control power dynamic, Enable=True: power on, Enable=False: power off*/
    return gcvSTATUS_OK;
}

gceSTATUS _Reset(IN gcsPLATFORM * Platform, IN gceCORE GPU)
{
    /*implemenet it to reset on NPU*/
    return gcvSTATUS_OK;
}

gceSTATUS _PutPower(IN gcsPLATFORM *Platform)
{
    /*implemenet it to power off NPU*/
    return gcvSTATUS_OK;
}

static gcsPLATFORM_OPERATIONS default_ops =
{
    .adjustParam    = _AdjustParam,
    .getPower       = _GetPower,
    .reset          = _Reset,
    .putPower       = _PutPower,
    .setPower       = _SetPower,
};

static gcsPLATFORM default_platform =
{
    .name = __FILE__,
    .ops  = &default_ops,
};

static struct platform_device *default_dev;
static const struct of_device_id galcore_dev_match[] = {
    {
        .compatible = "vivante, galcore"
    },
    { },
};

int gckPLATFORM_Init(struct platform_driver *pdrv, gcsPLATFORM **platform)
{
    pdrv->driver.of_match_table = galcore_dev_match;

    *platform = &default_platform;
    return 0;
}

int gckPLATFORM_Terminate(gcsPLATFORM *platform)
{
    if (default_dev) {
        platform_device_unregister(default_dev);
        default_dev = NULL;
    }
    return 0;
}

```

4. Compile the updated platform code files.
5. If the AXI SRAM physical address from the CPU view differs from that from the GPU view, use `getGPUPhysical()` to convert the CPU physical address into a GPU physical address in the AXI SRAM range.

4.2.3 Integrating Power Management

To ensure proper power-on and power-off procedure of Vivante IP, make sure that both Vivante and SoC software follow the power-on and power-off sequences described in this section.

Powering On Vivante IP

1. The SoC software requests a transition from the OFF power state to ON.
2. The Vivante software issues a callback to enable the Vivante clock and power.
3. The SoC software initiates a power-up sequence to the SoC controller hardware.
4. The SoC hardware de-asserts the power gate signal to turn on the power for the Vivante IP.
5. The SoC software returns a callback status to the Vivante software.
6. Upon a successful return, the Vivante software performs an AHB write that soft resets the core and AXI domains.
7. The Vivante software enables FE and waits for commission.

Powering Off Vivante IP

1. The SoC software issues a request for switching to the OFF power state, or the Vivante IP internal timer triggers a transition into the OFF state.
2. The Vivante software performs all the preliminary steps required for enabling the Vivante IP to enter the OFF state.
3. The Vivante software issues a callback to gate the Vivante clock and power.
4. The SoC software initiates a power-off sequence to the SoC controller hardware.
5. The SoC hardware asserts an isolation signal to turn on isolation from Vivante IP.
6. The SoC hardware asserts a power gate signal to turn off the power for the Vivante IP.
7. Upon return of hardware acknowledgement, the SoC software returns a callback status to the Vivante software.
8. Upon receipt of a successful callback, the Vivante software puts the Vivante IP into the "OFF" state.

4.2.3.1 Callback Functions for Power Management

You need to implement the following power management related callback functions in your platform-specific patch `hal/os/linux/kernel/platform/${soc_vendor}/`.

- `getPower`
- `putPower`
- `setPower`
- `setClock`
- `reset`

4.3 Loading the Unified Driver

You need to properly load the unified driver before networks can be run with it.

Prerequisites

Make sure that the unified driver is built, as instructed in [Section 4.2, Building the Unified Driver](#).

Procedure

1. Set insmod parameters according your actual environment.

The following example shows the insmod parameter settings in the case of a single VIP core:

```
INSMOD_ARGS="\
irqLine=10 \
registerMemBase=0xf0000000 \
registerMemSize=0x00020000 \
contiguousBase=0x10000000 \
contiguousSize=0x400000 \
showArgs=1 \
powerManagement=1 \
recovery=0 \
extSRAMBases=0xf0800000"
```

The following example shows the insmod parameter settings in the case of four VIP core:

```
INSMOD_ARGS="\
irqs=10,10,10,10 \
registerBases=0xf0000000,0xf1000000,0xf2000000,0xf3000000 \
registerSizes=0x2000,0x2000,0x2000,0x2000 \
contiguousBase=0x10000000 \
contiguousSize=0x400000 \
showArgs=1 \
powerManagement=1
physSize=0x80000000
recovery=0
extSRAMBases=0xf0800000"
```

2. Set the library path and run the insmod command as follows:

```
export VIVANTE_SDK_DIR=$AQROOT/build/sdk
export VIV_DRIVER_INC_DIR=$AQROOT/build/sdk/include
export LD_LIBRARY_PATH=$AQROOT/build/sdk/drivers
insmod $AQROOT/build/sdk/drivers galcore.ko $INSMOD_ARGS
```

In these commands, \$AQROOT indicates the driver path.

4.4 Enabling Network Profiling

If you are using Vivante GALVIP unified driver of a version earlier than 6.4.5, hardware profiling is disabled by default. To enable it, use the following procedure to set the `gpuProfiler` argument in the `galcore` module to 1.

Important: For driver versions 6.4.5 and later, hardware profiling is enabled by default. You can skip this procedure. The control argument is `vProfiler` instead of `gpuProfiler` in the `galcore` module.

Procedure

1. In the command line tool, run the `insmod galcore` command with `gpuProfiler` set to 1 as follows:

```
insmod /usr/lib/galcore.ko gpuProfiler=1 showArgs=1
```
2. Run the `demsg` command to show the arguments settings of the `galcore` module.
 - If the value of `gpuProfiler` is 1, hardware profiling has been enabled. Proceed to profiling configurations, as described in [Chapter 5, Environment Variables](#).
 - Otherwise, continue the following steps.
3. Open the platform code.
4. In the platform code, locate the `gpuProfiler` argument and set this argument to 1.
5. Proceed to profiling configurations, as described in [Chapter 5, Environment Variables](#).

4.5 Running a Network

This section describes how to run a network with the unified driver.

Prerequisites

Make sure that the unified driver is loaded, as instructed in [Section 4.3, Loading the Unified Driver](#).

Procedure

Use the following command to run the network:

```
./bin_r/<MODEL_DIR><QUANTIZED> <MODEL_DIR>_<QUANTIZED>.export.data <input tensor file name>
```

Result

An example of the network execution command and returned execution result is as follows:

```
./bin_r/lenetuint8 lenet_uint8.export.data iter_0_input_0_out0_1_1_28_28.tensor

Create Neural Network: 7ms or 7718us
Verify...
Verify Graph: 1805ms or 1805342us
Start run graph [1] times...
Run the 1 time: 4.28ms or 4283.14us
vxProcessGraph execution time:
Total 4.31ms or 4305.13us
Average 4.31ms or 4305.13us
--- Top5 ---
0: 0.999512
5: 0.000149
```

```
6: 0.000149
2: 0.000102
1: 0.000000
```

4.6 Developing Applications

4.6.1 Integrating Networks into Applications

4.6.1.1 Post-Processing Output Tensor Data

After network processing is completed, the output tensor data needs to be extracted from the graph and then further processed by a post-processing module in your application.

This section uses the mobilenet_ssd network as an example to describe how to pass output tensors to an application project. For the complete post-processing script, see the following file:

application_guide\mobilenet_ssd\mobilenet_ssd_uint8

Procedure

1. Include the post-processing files `ssd_post_process.c` and `ssd_post_process.h` in the application project and parse the output tensor data after the `vnn_ProcessGraph()` function as follows:

```
/* Verify graph */
status = vnn_VerifyGraph(graph);
TEST_CHECK_STATUS(status, final);

/* Process graph */
status = vnn_ProcessGraph( graph );
TEST_CHECK_STATUS( status, final );
vnn_get_ssd_result(graph, "mbox_priorbox_115.bin", (char*)image_name);
```

where `Mbox_priorbox_115.bin` is the priority box data generated by ACUTY.

2. Specify and pass the output tensors to the post-processing module as follows:

```
tensor_loc = vsi_nn_GetTensor(graph, graph->output.tensors[0]);
tensor_conf = vsi_nn_GetTensor(graph, graph->output.tensors[1]);
output_ptr_conf = (vx_float32*)GetDataFromTensor(graph, graph->output.tensors[1]);
output_ptr_loc = (vx_float32*)GetDataFromTensor(graph, graph->output.tensors[0]);
```

where:

- `output.tensors[0]` corresponds to the location tensor.
- `output.tensors[1]` corresponds to the confidence tensor.

For multi-output networks, make sure that the correspondence between output tensors and parameters is consistent with the network code and the original network.

3. Run the application after integration.

A returned result example is as follows:

```
start detect...
Detected objects:3
Class car label: 7, score: 0.999023, location: 0.607556,0.117253,0.883418,0.303419
Class dog label: 12, score: 0.995117, location: 0.180593,0.339222,0.415354,0.930478
Class bicycle label: 2, score: 0.786621, location: 0.069118,0.234948,0.731428,0.750805
```

4.6.1.2 Adding an OpenVX Kernel for Pre-Processing on PPU

The graph of a network can be sent to an OpenVX kernel for pre-processing on PPU. During pre-processing, the OpenVX kernel forms and processes a new graph from the received graph. For example, during pre-processing on the OpenVX kernel, the image can be scaled, the channel mean and scale can be reduced, and the image can be converted into a quantized tensor.

This section uses the mobilenet_ssd network as an example to describe how to implement an OpenVX kernel. For the complete kernel script, see `application guide\mobilenet_ssd\mobilenet_ssd_uint8_preprocess`.

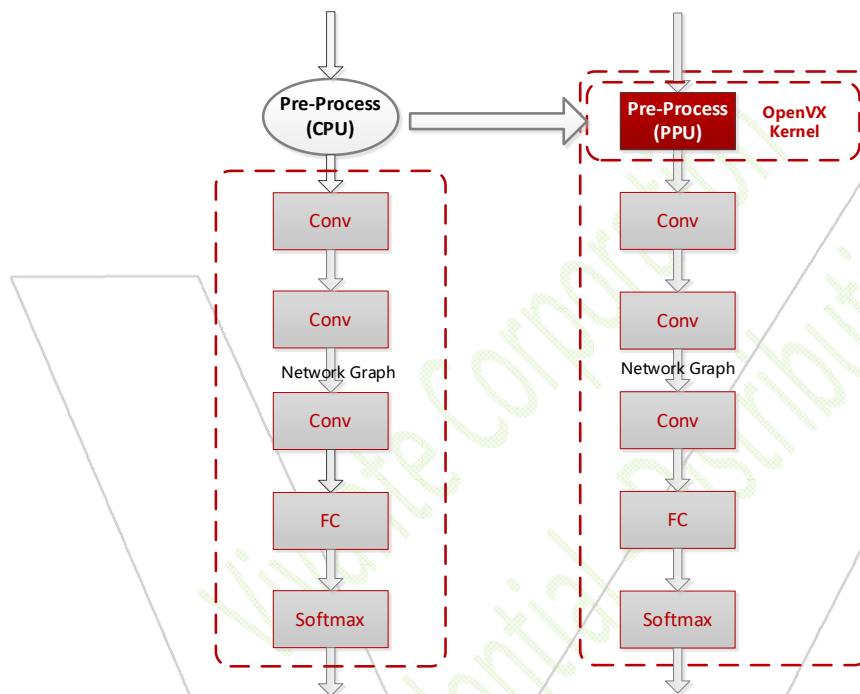


Figure 13. Adding an OpenVX Kernel

Note: For information about how to program PPU kernels, see the following documents:

- Vivante.VIP.A.Overview
- Vivante.VIP.Programming_Framework
- Vivante.VIP.Programming_EVIS2
- Vivante.VIP.DP.Instructions
- Vivante.VIP-VX.Introduction.to.EasyEVIS

Prerequisites

Make sure that a network can be properly run, as instructed in [Section 4.5, Running a Network](#).

Procedure

1. Integrate an OpenVX kernel.
 - a. Add the OpenVX kernel library RGBScaleToTensor to the application.
 - b. Pass an image to the OpenVX kernel and pre-process the image.

In the following example script, the 432x432 RGB input image is scaled to the network input size of 300x300.

Make sure that proper network input parameters are used as the output parameters of the OpenVX kernel.

When OpenVX constructs a graph, it rebuilds a new graph based on the data flow relationship between input and output.

```
const int width  = 432;
const int height = 432;
const int channel = 3;
const int dst_width  =300;
const int dst_height =300;
const float r_mean  = 127.5;
const float g_mean  = 127.5;
const float b_mean  = 127.5;
const float scale   = 1.0/127.5;
vx_int32 xRatio, yRatio;
vx_rectangle_t rect = {0, 0, width*channel, height};
vx_map_id map_id;
vx_imagepatch_addressing_t imgInfo = VX_IMAGEPATCH_ADDR_INIT;
void *imgData = NULL;
vsi_nn_tensor_t *tensor = vsi_nn_GetTensor( graph, graph->input.tensors[0]);
vx_uint8* bmpData = vnn_ReadJpegImage(image_name, width, height, channel);
/* write to file for nbg test */
FILE *fp = fopen("input_image_432x432x3.dat", "wb+");
if (fp != NULL)
    fwrite(bmpData, width*height*channel, sizeof(vx_uint8), fp);
fclose(fp);

input = vxCreateImage(graph->ctx->c, width*channel, height, VX_DF_IMAGE_U8);
vxMapImagePatch(input, &rect, 0, &map_id, &imgInfo, (void*)&imgData, VX_WRITE_ONLY, VX_MEMORY_TYPE_HOST,
0);
memcpy(imgData,bmpData,width*height*channel*sizeof(vx_uint8));
vxUnmapImagePatch(input, map_id);
if(bmpData)
    free(bmpData);
xRatio = (width << 15) / dst_width;
yRatio = (height << 15) / dst_height;
scaler = vxcBGRScaleToTensorNode(graph->g, input, xRatio, yRatio, 0, 0, r_mean,g_mean,b_mean, scale,
tensor->t);
```

2. Identify inputs and outputs for NBG generation.

If you need to use a network with a pre-processing node to generate NBG files in VivanteIDE or on an FPGA, the input and output parameters of the network need to be re-identified. This is because the network inputs and structure change after the OpenVX kernel.

In this example, the original network input is a tensor. After the OpenVX kernel is implemented, the network input becomes an image and a scaler node is added to the network. If the values of the scaler input parameters xRatio and yRatio need to be updated, you need to identify the parameters as inputs of the application. Otherwise, the xRatio and yRatio parameters are fixed to the values specified when the parameters are created.

```
uint32_t num_of_graph_inputs=1;
uint32_t num_of_graph_outputs=2;
vx_reference graph_inputs[1];
vx_reference graph_outputs[2];

vsi_nn_tensor_t      *output0,*output1;
graph_inputs[0] = (vx_reference)input;

output0 = vsi_nn_GetTensor(graph,graph->output.tensors[0]);
output1 = vsi_nn_GetTensor(graph,graph->output.tensors[1]);
graph_outputs[0] = (vx_reference)output0->t;
graph_outputs[1] = (vx_reference)output1->t;

status = vxIdentifyGraphInputsAndOutputs( graph->g,
                                           num_of_graph_inputs,
                                           graph_inputs,
                                           num_of_graph_outputs,
                                           graph_outputs );
```

4.6.1.3 Creating NBG Applications

This section provides an example script to show how to generate an NBG file from a unified network project and create an application from the NBG file. For the complete script, see the following file:

application_guide\mobilenet_ssd\mobilenet_ssd_uint8_preprocess_unify_nbg

By default, the generated NBG application does not include post-processing. You need to manually add code for post-processing. For example code, see other files in the mobilenet_ssd directory.

Make sure that the input and output parameters are created for the graph in the NBG application. You can query the parameter types and values from the original network or through the NBInfo tool. For information about how to use NBInfo for parameter query, see [Section 7.3, Querying the Input Parameters in an NBG File](#) and [Section 7.4, Querying the Output Parameters in an NBG File](#).

```
/*@input_0*/
width  = 432;
height = 432;
channel = 3;
input = vxCreateImage(context, width*channel,height,VX_DF_IMAGE_U8);
_CHECK_OBJ(input, exit);
/*@output_0*/
sizes[0]          = 7668;
sizes[1]          = 1;
ts_params.data_format = VX_TYPE_UINT8;
ts_params.num_of_dims = 2;
ts_params.sizes     = sizes;
ts_params.quant_format = VX_QUANT_AFFINE_SCALE;
ts_params.quant_data.affine.scale = 0.124951981;
ts_params.quant_data.affine.zeroPoint = 169;
output0 = vxCreateTensor2(context, &ts_params, sizeof(ts_params));
_CHECK_OBJ(output0, exit);
/*@output_1*/
sizes[0]          = 40257;
sizes[1]          = 1;
ts_params.data_format = VX_TYPE_FLOAT16;
ts_params.num_of_dims = 2;
```

```
ts_params.sizes = sizes;
ts_params.quant_format = VX_QUANT_NONE;
output1 = vxCreateTensor2(context, &ts_params, sizeof(ts_params));
_CHECK_OBJ(output1, exit);
```

4.6.2 Using Image/Tensor Handles

This section provides guidance on how to use handles for creation and control of images and tensors.

If an image or tensor object is created using `vxCreateImage()` or `vxCreateTensor()`, the object data in the data buffer cannot be directly accessed. You need to copy a view patch from/into the object or map a view patch out. However, this adds extra memory overhead.

Alternatively, you can create a reference to an image/tensor object that is externally allocated by using `vsi_nn_AddTensorFromHandle()` or by using `vxCreateImageFromHandle()` or `vxCreateTensorFromHandle()` with `VX_MEMORY_TYPE_HOST`. In this case, the pointer array is of the HOST type and is accessible to memory.

Table 10 describes the available handle APIs.

Note 1: The handle APIs are available only for hardware deployment on the FPGA. If you are working with VivanteIDE or vSimulator, use `vxCreateImage()` and `vxCreateTensor()` instead.

Note 2: OVXLIB APIs are recommended for tensor handles. However, OVXLIB does not provide handle APIs for images.

Table 10. Handle APIs

Object Type	API	API Type	Description
Image	<code>vxCreateImageFromHandle()</code>	Standard OpenVX APIs	Creates a reference to an image object that is externally allocated.
	<code>vxSwapImageHandle()</code>		Swaps the handle of an image previously created from a handle.
Tensor	<code>vxCreateTensorFromHandle()</code>	Vivante OpenVX-NN extension APIs	Creates a reference to a tensor object that is externally allocated.
	<code>vxSwapTensorHandle()</code>		Swaps the handle of a tensor previously created from a handle.
	<code>vsi_nn_AddTensorFromHandle()</code>	(Recommended) OVXLIB APIs	Creates a tensor from a memory handle and adds it to a graph.
	<code>vsi_nn_SwapHandle()</code>		Swaps a tensor handle of a tensor either created by an application or OVXLIB. This API requires three input parameters: <code>tensor</code> , <code>new_ptr</code> , and <code>old_ptr</code> . It sets the handle to <code>old_ptr</code> to read/write and updates the handle of tensor with the handle to <code>new_ptr</code> . For an application created tensor, make sure that the application properly maintains the tensor handle and manages the memory. Before freeing

Object Type	API	API Type	Description
			the data in the tensor handle, call this API as follows to get control of the handle: <code>vsi_nn_SwapHandle(tensor, NULL, &prev_ptr)</code> For an OVXLIB created tensor, do not free or write data to the tensor handle, which is maintained by OVXLIB. OVXLIB no longer maintains the tensor handle if this API is called for the tensor with <code>new_ptr</code> set to NULL. (Available from OVXLIB version 1.1.61)
	<code>vsi_nn_FlushHandle()</code>		Flushes a cache. This API must be called after data is written into a tensor handle.
	<code>vsi_nn_InvalidateHandle()</code>		Invalidates a cache. This API must be called before data is read from a tensor handle. (Available from OVXLIB version 1.1.61)
	<code>vsi_nn_GetTensorHandle()</code>		Gets the handle of the tensor.

4.6.2.1 Allocating Aligned Memory

This section provides a sample script to show how to allocate an aligned buffer by using handles.

Note: For direct access to allocated memory, the alignment requirements are that the start address and the block size must be 64-byte aligned. The VIP kernel driver flushes the cache in 64 KB units. If the size of the allocated memory is not 64 KB-aligned, data may be unexpectedly flushed.

```
#define HANDLE_ADDR_ALIGN_START_SIZE 64
#define HANDLE_ADDR_ALIGN_BLOCK_SIZE 64

typedef struct
{
    uint8_t* raw_addr;
} aligned_header;

uint8_t * vsi_nn_MallocAlignedBuffer
(
    uint32_t mem_size,
    uint32_t align_start_size,
    uint32_t align_block_size
)
{
    uint32_t sz = 0;
    long temp = 0;
    uint8_t* raw_addr = NULL;
    uint8_t* p = NULL;
    uint8_t* align_addr = NULL;
    aligned_header* header = NULL;
```

```

sz = sizeof(aligned_header) + mem_size + align_start_size + align_block_size;
raw_addr = (uint8_t *)malloc( sz * sizeof( uint8_t ) );
memset(raw_addr, 0, sizeof( uint8_t ) * sz);
p = raw_addr + sizeof(aligned_header);

temp = (long)(p) % align_start_size;
if (temp == 0)
{
    align_addr = p;
}
else
{
    align_addr = p + align_start_size - temp;
}
header = (aligned_header*)(align_addr - sizeof(aligned_header));
header->raw_addr = raw_addr;
return align_addr;
}/* vsi_nn_MallocAlignedBuffer() */

void vsi_nn_FreeAlignedBuffer
(
    uint8_t* handle
)
{
    aligned_header* header = NULL;
    header = (aligned_header*)(handle - sizeof(aligned_header));
    free(header->raw_addr);
}/* vsi_nn_FreeAlignedBuffer() */

```

4.6.2.2 Creating Images/Tensors from Handles

The sample scripts in this section illustrate how to create images and tensors from handles.

Creating an Image from a Handle

```

vx_context context = NULL;
vx_image image = NULL;
vx_uint8* data_buf = NULL;
vx_imagepatch_addressing_t imgInfo = {WIDTH*DEPTH, HEIGHT, 1, WIDTH*DEPTH,
                                       VX_SCALE_UNITY, VX_SCALE_UNITY, 1, 1};

context = vxCreateContext();
data_buf = (vx_uint8*)vsi_nn_MallocAlignedBuffer(
    SIZE*sizeof(vx_uint8),
    HANDLE_ADDR_ALIGN_START_SIZE,
    HANDLE_ADDR_ALIGN_BLOCK_SIZE
);
memset(data_buf,0,SIZE*sizeof(vx_uint8));
image = vxCreateImageFromHandle(
    context,
    VX_DF_IMAGE_U8,
    &imgInfo,
    (void **)&data_buf,
    VX_MEMORY_TYPE_HOST
);

```

Creating a Tensor from a Handle

```
#define NEW_HANDLE_TENSOR(_id, _attr, _dtype, data) do {\
    _attr.vtl = FALSE;\
    _attr.is_const = FALSE;\
    _attr.dtype.vx_type = _dtype;\
    _id = vsi_nn_AddTensorFromHandle( graph, VSI_NN_TENSOR_ID_AUTO,\
        &_attr, data );\
    if( VSI_NN_TENSOR_ID_NA == _id ) {\
        goto error;\
    }\
} while(0)
#endif

vsi_status          status = VSI_FAILURE;
vsi_nn_context_t    ctx = NULL;
vsi_nn_tensor_id_t  norm_tensor[NET_NORM_TENSOR_NUM];
vsi_nn_tensor_attr_t attr;
vx_uint8* data_buf = NULL;

ctx = vsi_nn_CreateContext();
graph = vsi_nn_CreateGraph( ctx, NET_TOTAL_TENSOR_NUM, NET_NODE_NUM );
memset( &attr, 0, sizeof( attr ) );
data_buf = vnn_MallocAlignedBuffer(
    WIDTH*HEIGHT*DEPTH*sizeof(vx_uint8),
    HANDLE_ADDR_ALIGN_START_SIZE,
    HANDLE_ADDR_ALIGN_BLOCK_SIZE
);
memset(data_buf,0,SIZE*sizeof(vx_uint8));
NEW_HANDLE_TENSOR(norm_tensor[0], attr, VSI_NN_TYPE_UINT8, data_buf);
```

4.6.2.3 Updating Handle Data

If the image/tensor is the input of the graph, you need to prepare the input data before processing the graph. This means that the input data needs to be updated to the handle buffer before the image/tensor is created. However, in practice, you need to update the data buffer whenever a new frame arrives.

In this case, you can use the `vxSetParameterByIndex()` API and the swap handle APIs listed in [Table 10](#) to update handle data without the need to re-create images/tensors or re-verify the graph.

`vxSetParameterByIndex()`

The `vxSetParameterByIndex()` API updates the data in the handle buffer from which the image/tensor is created.

For example, the following script shows how to update the image data in the buffer of the `data_buf` handle. For information about the `data_buf` handle, see [Section 4.6.2.2, Creating Images/Tensors from Handles](#).

```
vx_node node = vxCopyNode(graph,(vx_reference)image,(vx_reference)image_out);
status = vxVerifyGraph(graph);
//re-write new data to the handle buffer
generate_random_input_data(data_buf,SIZE*sizeof(vx_uint8));

//update new data to the input of node
status = vxSetParameterByIndex(node,0,(vx_reference)image);
status = vxProcessGraph(graph);
```

Swap Handle APIs

The `vxSwapImageHandle()` and `vxSwapTensorHandle()` APIs can either update the data in the existing handle buffer or replace the existing handle buffer with a new aligned one.

For example, the following script shows how to update the handle data by replacing the existing handle buffer with a newly allocated aligned buffer.

```
vx_node node = vxCopyNode(graph, (vx_reference)image, (vx_reference)image_out);
status = vxVerifyGraph(graph);

//allocate new aligned data buffer
vx_uint8* data_buf_new = (vx_uint8*)vsi_nn_MallocAlignedBuffer(
    SIZE*sizeof(vx_uint8),
    HANDLE_ADDR_ALIGN_START_SIZE,
    HANDLE_ADDR_ALIGN_BLOCK_SIZE
);

//write aligned data buffer
generate_random_input_data(data_buf_new, SIZE*sizeof(vx_uint8));

// swap the new aligned data buffer to the input image
status = vxSwapImageHandle(image, (void*)&data_buf_new, NULL, 1);
status = vxProcessGraph(graph);
```

4.6.2.4 Example

This section provides an example script, which shows the entire process of using image/tensor handles, covering from allocating aligned memory, creating images/tensors from handles, to updating handle data.

```
#include <VX/vx.h>
#include <VX/vxu.h>
#include <VX/vx_api.h>
#include <VX/vx_khr_cnn.h>
#include <VX/vx_lib_extras.h>
#include <VX/viv_nn_compatibility.h>

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <errno.h>
#include <math.h>
#include <malloc.h>
#include <time.h>

#define TEST_CHECK_STATUS( stat, lbl ) do {\
    if( VX_SUCCESS != stat ) {\
        printf("CHECK STATUS %d", __LINE__);\
        goto lbl;\
    }\
} while(0)

#define TEST_CHECK_OBJ( obj, lbl ) do {\
    if( NULL == obj ) {\
        printf("CHECK OBJECT %d", __LINE__);\
```



```

        goto lbl;\
    }\
} while(0)

#define HANDLE_ADDR_ALIGN_START_SIZE 64
#define HANDLE_ADDR_ALIGN_BLOCK_SIZE 64
#define WIDTH 640
#define HEIGHT 480
#define DEPTH 3
#define SIZE WIDTH*HEIGHT*DEPTH

typedef struct
{
    uint8_t* raw_addr;
} aligned_header;

uint8_t * vsi_nn_MallocAlignedBuffer
(
    uint32_t mem_size,
    uint32_t align_start_size,
    uint32_t align_block_size
)
{
    uint32_t sz = 0;
    long temp = 0;
    uint8_t* raw_addr = NULL;
    uint8_t* p = NULL;
    uint8_t* align_addr = NULL;
    aligned_header* header = NULL;

    sz = sizeof(aligned_header) + mem_size + align_start_size + align_block_size;
    raw_addr = (uint8_t *)malloc( sz * sizeof( uint8_t ) );
    memset(raw_addr, 0, sizeof( uint8_t ) * sz);
    p = raw_addr + sizeof(aligned_header);

    temp = (long)(p) % align_start_size;
    if (temp == 0)
    {
        align_addr = p;
    }
    else
    {
        align_addr = p + align_start_size - temp;
    }
    header = (aligned_header*)(align_addr - sizeof(aligned_header));
    header->raw_addr = raw_addr;
    return align_addr;
}/* vsi_nn_MallocAlignedBuffer() */

void vsi_nn_FreeAlignedBuffer
(
    uint8_t* handle
)
{
    aligned_header* header = NULL;
    header = (aligned_header*)(handle - sizeof(aligned_header));
    free(header->raw_addr);
}/* vsi_nn_FreeAlignedBuffer() */

```

```

void generate_random_input_data(uint8_t* data_buf,int buf_size)
{
    uint8_t *p_buff = NULL;
    int i = 0;
    srand((int)time(NULL));
    p_buff = data_buf;
    for (i = 0; i < buf_size; i++)
    {
        p_buff[i] = rand()%255;
    }
}

vx_status check_result(uint8_t* data_in,uint8_t* data_out,int buf_size)
{
    int i = 0;
    for (i = 0; i < buf_size; i++)
    {
        if(data_in[i]!=data_out[i])
            return VX_FAILURE;
    }
    return VX_SUCCESS;
}

void image_handle_sample()
{
    vx_status status = VX_SUCCESS;
    vx_context context = NULL;
    vx_graph graph = NULL;
    vx_image image_in = NULL;
    vx_image image_out = NULL;
    vx_node node = NULL;
    vx_uint8* handle_buf_in = NULL;
    vx_uint8* data_buf_in = NULL;
    vx_uint8* handle_buf_out = NULL;
    vx_bool swap_new_handle = vx_true_e;
    vx_imagepatch_addressing_t imgInfo = {WIDTH*DEPTH, HEIGHT, 1, WIDTH*DEPTH, VX_SCALE_UNITY, VX_SCALE_UNITY, 1, 1};

    context = vxCreateContext();
    TEST_CHECK_OBJ(context,exit);
    graph = vxCreateGraph(context);
    TEST_CHECK_OBJ(graph,exit);

    handle_buf_in = (vx_uint8*)vsi_nn_MallocAlignedBuffer(
        SIZE*sizeof(vx_uint8),
        HANDLE_ADDR_ALIGN_START_SIZE,
        HANDLE_ADDR_ALIGN_BLOCK_SIZE
    );
    data_buf_in = (vx_uint8*)vsi_nn_MallocAlignedBuffer(
        SIZE*sizeof(vx_uint8),
        HANDLE_ADDR_ALIGN_START_SIZE,
        HANDLE_ADDR_ALIGN_BLOCK_SIZE
    );
    handle_buf_out = (vx_uint8*)vsi_nn_MallocAlignedBuffer(
        SIZE*sizeof(vx_uint8),
        HANDLE_ADDR_ALIGN_START_SIZE,

```

```

        HANDLE_ADDR_ALIGN_BLOCK_SIZE
    );
    // init handles
    memset(handle_buf_in,0,SIZE*sizeof(vx_uint8));
    memset(data_buf_in,0,SIZE*sizeof(vx_uint8));
    memset(handle_buf_out,0,SIZE*sizeof(vx_uint8));

    //create input and output image
    image_in = vxCreateImageFromHandle(
        context,
        VX_DF_IMAGE_U8,
        &imgInfo,
        (void **)&handle_buf_in,
        VX_MEMORY_TYPE_HOST
    );
    image_out = vxCreateImageFromHandle(
        context,
        VX_DF_IMAGE_U8,
        &imgInfo,
        (void **)&handle_buf_out,
        VX_MEMORY_TYPE_HOST
    );
    //create node
    node = vxCopyNode(graph,(vx_reference)image_in,(vx_reference)image_out);
    TEST_CHECK_OBJ(node,exit);

    //verify graph
    status = vxVerifyGraph(graph);
    TEST_CHECK_STATUS(status,exit);

    //update data buffer
    if(swap_new_handle)
    {
        //swap to new handle data_buf_in
        generate_random_input_data(data_buf_in,SIZE*sizeof(vx_uint8));
        status = vxSwapImageHandle(image_in,(void **)&data_buf_in, (void **)&data_buf_in,1);
        TEST_CHECK_STATUS(status,exit);
    }
    else
    {
        //just update the handle buffer which create image/tensor from
        generate_random_input_data(handle_buf_in,SIZE*sizeof(vx_uint8));
        status = vxSetParameterByIndex(node,0,(vx_reference)image_in);

        //also can use vxSwapImageHandle to update the handle buffer data
        //status = vxSwapImageHandle(image_in,(void **)&handle_buf_in,NULL,1);
        TEST_CHECK_STATUS(status,exit);
    }

    status = vxProcessGraph(graph);
    TEST_CHECK_STATUS(status,exit);

    if(swap_new_handle)
        status = check_result(data_buf_in,handle_buf_out,SIZE*sizeof(vx_uint8));
    else
        status = check_result(handle_buf_in,handle_buf_out,SIZE*sizeof(vx_uint8));
    if(status == VX_SUCCESS)
        printf("Check result pass.\n");

```

```

else
    printf("Check result fail.\n");

exit:
    if(handle_buf_in) vsi_nn_FreeAlignedBuffer(handle_buf_in);
    if(data_buf_in) vsi_nn_FreeAlignedBuffer(data_buf_in);
    if(handle_buf_out) vsi_nn_FreeAlignedBuffer(handle_buf_out);
    if(image_in) vxReleaseImage(&image_in);
    if(image_out) vxReleaseImage(&image_out);
    if(node) vxReleaseNode(&node);
    if(graph) vxReleaseGraph(&graph);
    if(context) vxReleaseContext(&context);
    return;
}

int main()
{
    image_handle_sample();
    return 0;
}

```

5 Environment Variables

This section describes the environment variables used for the unified driver on FPGA and in VivanteIDE.

5.1 Environment Variables

5.1.1 Environment Variables for ACUITY Generated Application Configuration

The following environment variables are used for configuration of network applications generated by ACUITY Toolkit.

Table 11. Environment Variables for ACUITY Generated Application Configuration

Environment Variable	Value	Default	Description
VNN_LOOP_TIME	[0,N]	1	Specifies the number of times to execute the graph. For tests, specify multiple times to obtain the average execution time. For examples of how to use this variable, see Section 5.2.3, Running a Network Multiple Times .
VSI_USE_IMAGE_PROCESS	[0,1]	0	Specifies whether to allow image pre-processing and post-processing with PPU nodes. Set this variable to one of the following values: 0: disallows the pre-processing and post-processing. 1: allows the pre-processing and post-processing.
VSI_SAVE_FILE_TYPE	[0,2]	0	Specifies the type of the file to which the output tensor is saved. Set this variable to one of the following values: 0: float32 text file 1: qtensor file 2: binary file

5.1.2 Environment Variables for OVXLIB Configuration

The following environment variables are used for configuration of the OVXLIB library.

Table 12. Environment Variables for OVXLIB Configuration

Environment Variable	Value	Default	Description
VSI_NN_LOG_LEVEL	[0,4]	2	Specifies whether to print OVXLIB logs. Set this variable to one of the following values: 0: does not print OVXLIB logs. 1: prints OVXLIB logs of the error level. 2: prints OVXLIB logs of the error and warning levels. 3: prints OVXLIB logs of the error, warning, and information levels. 4: prints OVXLIB logs of the error, warning, information, and debug levels. For examples of how to use this variable, see Section 5.2.1, Disabling OVXLIB Log Printing and Section 5.2.2, Enabling OVXLIB Log Printing.
USER_VX_SOURCE_PATH	String	NULL	Specifies the VX file path of the user defined kernel. This variable is available only for custom layers.

5.1.3 Environment Variables for OpenVX Unified Driver Configuration

The following environment variables are used for configuration of the Vivante GALVIP unified driver for OpenVX.

Table 13. Environment Variables for OpenVX Unified Driver Configuration

Environment Variable	Value	Default	Description
VIV_VX_DEBUG_LEVEL	[0,1]	0	Specifies whether to print the driver log. Set this variable to one of the following values: 0: does not print the driver log. 1: prints the driver log. This variable can used together with other environment variables to control whether to print different types of driver logs. For examples of how to use this variable, see Section 5.2.4, Enabling Driver Log Printing, Section 5.2.5, Profiling a Network, and Section 5.2.6, Profiling Layers.

Environment Variable	Value	Default	Description
CNN_PERF	[0,1]	0	Specifies whether to print the execution time of each layer. The default value is 1 if VIV_VX_PROFILE is set to 1 and 0 in other cases. To print the per-layer execution time, set both VIV_VX_DEBUG_LEVEL and CNN_PERF to 1. For examples of how to use this variable, see Section 5.2.5, Profiling a Network and Section 5.2.6, Profiling Layers.
NN_LAYER_DUMP	[0,1]	0	Specifies whether to dump tensors. Set this variable to one of the following values: 0: does not dump tensors. 1: dumps tensor data of all layers and user nodes. If tensor dumping is enabled, SWTILING is automatically disabled. After network execution, a folder named LayerDump is generated, which stores the data of all intermediate layers. Tensor dumping is useful for debugging when the network output does not match your expectation. The output of each layer is saved as a file with the same name and format as those VivanteIDE used. For information about how to compare the dumped data with golden data, see Section 3.1.6, Comparing Dump Results.
NN_LAYER_DUMP_QTENSOR	[0,1]	0	Specifies whether to dump tensors in the float32 or quantized format. Set this variable to one of the following values: 0: float32 1: quantized This variable is valid only if NN_LAYER_DUMP is set to 1.
VIV_VX_PROFILE	[0,2]	0	Specifies whether to print the VIP performance log. The VIP performance log includes AXI read/write bandwidth, DDR read/write bandwidth, and consumed VIP cycles of each node. 0: does not print the VIP performance log. 1: prints the per-layer VIP performance log. 2: prints the per-network VIP performance log. If this variable is set to 1 or 2, VIV_VX_DEBUG_LEVEL must be set to 1 to enable log printing. For examples of how to use this variable, see Section 5.2.5, Profiling a Network and Section 5.2.6, Profiling Layers.

Environment Variable	Value	Default	Description
VIV_MEMORY_PROFILE	[0,1]	0	Specifies whether to print the memory footprint of the application. Set this variable to one of the following values: 0: does not print the memory footprint. 1: prints the memory footprint. For examples of how to use this variable, see Section 5.2.7, Profiling Memory Usage.
VIV_VX_ENABLE_SWTILING_PHASE1	[0,1]	1	Specifies whether to enable all SWTILING. Set this variable to one of the following values: 0: disables all SWTILING. 1: enables all SWTILING.
VIV_VX_ENABLE_SAVE_NETWORK_BINARY	[0,1]	0	Specifies whether to save the binary graph. Set this variable to one of the following values: 0: does not save the binary graph. 1: saves the binary graph to an NBG file. If this variable is set to 1, an NBG file (for example, <code>network_binary_pid-1495_tid-1261004544.nb</code>) is generated after network execution.
VIV_VX_NBG_NAME	String	NULL	Specifies the name of the NBG network, not the file name.
VIV_VX_SAVE_NETWORK_BINARY_PATH	String	NULL	Specifies the path to save the NBG network. By default, the NBG network is saved to the current work directory.
VIV_VX_ENABLE_CACHE_GRAPH_BINARY	[0,1]	0	Specifies whether to enable graph caching. Set this variable to one of the following values: 0: disables graph caching. 1: enables graph caching. When graph caching is enabled, the system checks whether there is a cached NBG file for a network before running it. - If an NBG file is already cached for the network, the system loads the NBG file for running the network. This reduces the network loading time. - In the case of no such NBG file cached, the system generates an NBG file for caching. For examples of how to use this variable, see Section 5.2.3, Running a Network Multiple Times.
VIV_VX_CACHE_BINARY_GRAPH_MAX_SIZEKB	[1,N]	524288 (= 512 MB)	Specifies the maximum size of a cached NBG file, in kilobytes.

Environment Variable	Value	Default	Description
VIV_VX_CACHE_BINARY_GRAPH_MAX_COUNT	[1,N]	5	Specifies the maximum number of cached NBG files.
VIV_VX_CACHE_BINARY_GRAPH_DIR	String	NULL	Specifies the path of the directory to save the cached NBG files. By default, the cached NBG files are saved in the current work directory.
VIV_VX_ENABLE_TP	[0,1]	1	Specifies whether to enable the TP engine. Set this variable to one of the following values: 0: disables the TP engine. 1: enables the TP engine.
VIV_VX_ENABLE_TP_SINGLE_FC	[0,1]	1	Specifies whether to enable the single FC function of the TP engine. Set this variable to one of the following values: 0: disables the single FC function. 1: enables the single FC function.
VIV_VX_ENABLE_TP_MAX_POOLING	[0,1]	1	Specifies whether to enable the max pooling function of the TP engine. Set this variable to one of the following values: 0: disables the max pooling function. 1: enables the max pooling function.
VIV_VX_ENABLE_TP_ROI_POOLING	[0,1]	1	Specifies whether to enable the ROI pooling function of the TP engine. Set this variable to one of the following values: 0: disables the ROI pooling function. 1: enables the ROI pooling function.
VIV_VX_DISABLE_TP_NN_EVIS	[0,1]	0	Specifies whether to disable NN, TP, and EVIS of the PPU engine. Set this variable to one of the following values: 0: enables NN, TP, and EVIS. 1: disables NN, TP, and EVIS.
VIV_VX_DISABLE_TP_NN	[0,1]	0	Specifies whether to disable the NN and TP engines. Set this variable to one of the following values: 0: enables the NN and TP engines. 1: disables the NN and TP engines.

Environment Variable	Value	Default	Description
VIV_VX_ENABLE_SHADER	[0,1]	1	Specifies whether to enable the PPU engine. Set this variable to one of the following values: 0: disables the PPU engine. 1: enables the PPU engine.
USE_RPN_MODE	[0,2]	0	Specifies whether to run the RPN layer on the PPU or CPU. Set this variable to one of the following values: 0: runs the RPN layer on the PPU and CPU. 1: runs the RPN layer on the CPU. 2: runs the RPN layer on the PPU.
VIV_VX_ENABLE_MULTIVIP_COMBINED	[0,1]	1	Specifies whether to enable multi-VIP combination. Set this variable to one of the following values: 0: disables multi-VIP combination. 1: enables multi-VIP combination.
VIV_MGPU_AFFINITY	0 1:n	0	Specifies whether to enable a VIP core to work in independent mode. Set this variable to one of the following values: 0: All VIP cores work in multi-VIP combination mode. 1:n: VIP core <i>n</i> works in independent mode. Other VIP cores work in multi-VIP combination mode. For examples of how to use this variable, see Section 5.2.9, Configuring Multi-VIP Scenarios .
VIV_OVX_USE_MULTI_DEVICE	0 1:n	0	Specifies whether to enable the multi-device mode in multi-VIP scenarios. Set this variable to one of the following values: 0: disables the multi-device mode. 1:n: enables each device to correspond to <i>n</i> VIP cores. This variable is valid only if VIV_MGPU_AFFINITY is set to 1:0. For examples of how to use this variable, see Section 5.2.9, Configuring Multi-VIP Scenarios .
VIV_VX_ENABLE_GRAPH_TRANSFORM	-closeAll		Disables all optimization features. The following options are valid only if -closeAll is not specified.

Environment Variable	Value	Default	Description
	-Merge:0 1	-Merge:1	Specifies whether to merge the operations as follows to the NN path: • Merge conv + relu + pool. • Merge fc + relu. Set this option to one of the following values: • 0: disables the operation merging. • 1: enables the operation merging.
	-Dump		Dumps the topology graph per optimization feature. If the -Dump option is specified, then after network execution: • The original graph is saved in <code>before_optimization_topology.json</code>. • The final optimized graph is saved in <code>final_graph_topology.json</code>. • Other files stores intermediate graphs.
	-ConvertBatchFC: 0 1 2	-ConvertBatchFC:1	Specifies the batch FC conversion method. Set this option to one of the following values: • 0: does not covert batch FC. • 1: converts batch FC into transpose + ConvolutionReluPoolingLayer2 + transpose. • 2: converts batch FC into GEMM by offline weight transpose.
	- ConvertAvgPool:0 1	- ConvertAvgPool:1	Specifies whether to convert the average pool operation. Set this option to one of the following values: • 0: does not converts average pool. • 1: converts average pool into depth-wise conv2d when the NN limitation is met.
	-avgPool1x1conv:0 1	-avgPool1x1conv:1	Specifies whether to replace the avg pool + 1x1 conv2d operation with conv2d. Set this option to one of the following values: • 0: does not replace avg pool + 1x1 conv2d. • 1: uses a conv2d operation with the kernel value equal to $\text{org_kernel_value} \times (1/\text{pool_size}[0] \times \text{pool_size}[1])$ to replace avg pool + 1x1 conv2d.

Environment Variable	Value	Default	Description
	-unrollDWConv:0 1	-unrollDWConv:1	Specifies whether to roll the depthwise conv operation back to common convolution if the hardware does not support depthwise conv. Set this option to one of the following values: 0: does not roll the depthwise conv operation back to common convolution. 1: rolls the depthwise conv operation back to common convolution.
	-tensorAdd:0 1	-tensorAdd:1	Specifies whether to optimize the elementwise add operation by overlaying the two inputs on the channel dimension and performing conv2d with a special 1x1 kernel. Set this option to one of the following values: 0: disables the optimization. 1: enables the optimization.
	-add2convadd: 0 1	-add2convadd: 1	Specifies whether to optimize the elementwise add operation by inserting dummy conv2d before the elementwise add and then merging conv2d + add into vxConvolutionReluPoolingAddLayer2. Set this option to one of the following values: 0: disables the optimization. 1: enables the optimization. The optimization is available only if the hardware supports vxConvolutionReluPoolingAddLayer2.
	-eltwiseOp:0 1	-eltwiseOp:1	Specifies whether to enable shape optimization for add and subtract operations. Set this option to one of the following values: 0: disables the shape optimization. 1: enables the shape optimization.
	-swap:0 1	-swap:1	Specifies whether to swap the order of leakyRelu + maxPool. Set this option to one of the following values: 0: does not swap the order. 1: swaps the order of leakyRelu + maxPool.
	-deleteReshape:0 1	deleteReshape:1	Specifies whether to replace the reshape node with tensorview. Set this option to one of the following values: 0: does not replace the reshape node. 1: replaces the reshape node with tensorview.

Environment Variable	Value	Default	Description
	-alignNMConv:0 1	-alignNMConv:1	Specifies whether to pad MxN conv to MxM in V7. Set this option to one of the following values: 0: does not pad MxN conv to MxM. 1: pads MxN conv to MxM.
	-conv2fc:0 1	-conv2fc:1	Specifies whether to convert convolution to FC. Set this option to one of the following values: 0: does not convert convolution. 1: converts convolution to FC.
	-mergeTranspose:0 1	-mergeTranspose:1	Specifies whether to merge multiple continuous transpose nodes into one node. Set this option to one of the following values: 0: does not merge multiple continuous transpose nodes. 1: merges multiple continuous transpose nodes into one node.
	-padConv:0 1	-padConv:1	Specifies whether to optimize conv2d by inserting a pad node before conv2d with the pad_size beyond the hardware limitation and then setting pad_size of conv2d to 0. Set this option to one of the following values: 0: disables the optimization. 1: enables the optimization.
	-conv3dto2d:0 1	-conv3dto2d:1	Specifies whether to convert conv3d into grouped conv2d. Set this option to one of the following values: 0: does not convert conv3d. 1: converts conv3d into grouped conv2d.
	-convdataconvert:0 1	-convdataconvert:1	Specifies whether to merge DataConvert after conv2d. Set this option to one of the following values: 0: does not merge DataConvert after conv2d. 1: merges DataConvert after conv2d.
	-gap2fc:0 1	-gap2fc:0	Specifies whether to convert GlobalAvgPool into FC. Set this option to one of the following values: 0: does not convert GlobalAvgPool. 1: converts GlobalAvgPool into FC.

Environment Variable	Value	Default	Description
	-fc2conv:0 1 2	-fc2conv:1	Specifies the FC conversion method. Set this option to one of the following values: 0: does not convert FC. 1: converts FC into convolution. 2: forcibly converts FC into convolution.
	-add2bias:0 1	-add2bias:1	Specifies whether to merge add into convolution bias. Set this option to one of the following values: 0: does not merge add into convolution bias. 1: merges add into convolution bias.
	-muladd2bn:0 1	-muladd2bn:1	Specifies whether to convert multiply + add into batch norm. Set this option to one of the following values: 0: does not convert multiply + add. 1: converts multiply + add into batch norm.
	-fusebn2conv:0 1	-fusebn2conv:1	Specifies whether to fuse batch norm to convolution. Set this option to one of the following values: 0: does not fuse batch norm to convolution. 1: fuses batch norm to convolution.
	-batchGemmtoFC:0 1	-batchGemmtoFC:1	Specifies whether to convert batch GEMM into FC. Set this option to one of the following values: 0: does not convert batch GEMM. 1: converts batch GEMM into FC.
	-resize2deconv:0 1	-resize2deconv:0	Specifies whether to convert resize into deconvolution. Set this option to one of the following values: 0: does not convert resize. 1: converts resize into deconvolution.
	-mean2dwconv:0 1	-mean2dwconv:1	Specifies whether to convert mean into depth-wise convolution. Set this option to one of the following values: 0: does not convert mean. 1: converts mean into depth-wise convolution.

Environment Variable	Value	Default	Description
	-fusedPad:0 1	-fusedPad:1	Specifies whether to fuse a pad operation before conv2d into the input of conv2d, if the hardware supports the pad size. Set this option to one of the following values: 0: does not perform the fuse. 1: fuses a pad operation before conv2d into the input of conv2d.
	-fusemul2conv:0 1	-fusemul2conv:1	Specifies whether to fuse the constant value into weight and bias when an input of convolution + multiply is a single pixel constant tensor. Set this option to one of the following values: 0: does not perform the fuse. 1: fuses the constant value into weight and bias when the multiplier is a single pixel constant tensor.
	-mul2conv:0 1	-mul2conv:1	Specifies whether to optimize the elementwise multiply operation by inserting dummy conv2d before the elementwise multiply and then merging conv2d + multiply. Set this option to one of the following values: 0: disables the optimization. 1: enables the optimization. The optimization is available only if the hardware supports conv2d + multiply merging.
	-convmul:0 1	-convmul:1	Specifies whether to merge conv2d + multiply into vxConvolutionReluPoolingMultiplyLayer2. Set this option to one of the following values: 0: does not merge conv2d + multiply. 1: merges conv2d + multiply into vxConvolutionReluPoolingMultiplyLayer2. The merge is available only if the hardware supports the vxConvolutionReluPoolingMultiplyLayer2.
	-convadd:0 1	-convadd:1	Specifies whether to merge conv2d + add (elementwise) into vxConvolutionReluPoolingAddLayer2. Set this option to one of the following values: 0: does not merge conv2d + add. 1: merge conv2d + add into vxConvolutionReluPoolingAddLayer2. The merge is available only if the hardware supports the related feature.

Environment Variable	Value	Default	Description
	-padDilationConv:0 1	-padDilationConv:1	Specifies whether to dilate the kernel by padding 0 and reset dilation parameters. Set this option to one of the following values: 0: disables the dilation. 1: enables the dilation.
	-splitMaxpool:0 1	-splitMaxpool:0	Specifies whether to split a large kernel into small kernels if the original kernel size beyond the hardware limitation. Set this option to one of the following values: 0: does not split a large kernel. 1: splits a large kernel.
	-pad1dMaxpool:0 1	-pad1dMaxpool:1	Specifies whether to pad 1d max pool to 3x3 or 2x2 so that the node can run on TP. Set this option to one of the following values: 0: does not pad 1d max pool to 3x3 or 2x2. 1: pads 1d max pool to 3x3 or 2x2.
	-maxpool2conv:0 1	-maxpool2conv:1	Specifies whether to perform optimization by inserting 1x1 depth-wise convolution before pooling and then merging the two operations into ConvolutionReluPoolingLayer2. Set this option to one of the following values: 0: disables the optimization. 1: enables the optimization.
	-war1x1x1weight:0 1	-war1x1x1weight:1	Specifies whether to convert the 1x1x1 kernel into 2x2x1 by padding 0. Set this option to one of the following values: 0: does not convert the 1x1x1 kernel. 1: converts the 1x1x1 kernel into 2x2x1 by padding 0.
	-ConvertActivation:0 1	-ConvertActivation:1	Specifies whether to convert activation into ConvolutionReluPoolingLayer2. Set this option to one of the following values: 0: does not convert activation. 1: converts activation into ConvolutionReluPoolingLayer2. The conversion is available only if the hardware supports the related feature.

Environment Variable	Value	Default	Description
	-mergeConstMul:0 1	-mergeConstMul:1	Specifies whether to merge single pixel constant input into the scale parameter in the output of ConvolutionReluPoolingLayer2 as follows: $\text{output_scale} = \text{output_scale} / \text{const_val}$ Set this option to one of the following values: 0: does not merge single pixel constant input into the scale parameter. 1: merges single pixel constant input into the scale parameter.
	-transposeConv:0 1	-transposeConv:1	Specifies whether to merge head transpose with ConvolutionReluPoolingLayer2. Set this option to one of the following values: 0: does not merge head transpose with ConvolutionReluPoolingLayer2. 1: merges head transpose with ConvolutionReluPoolingLayer2. The merge is available only if the hardware supports the related feature.
	-dataconvertconv:0 1	-dataconvertconv:1	Specifies whether to merge head DataConvert with ConvolutionReluPoolingLayer2. Set this option to one of the following values: 0: does not merge head DataConvert with ConvolutionReluPoolingLayer2. 1: merges head DataConvert with ConvolutionReluPoolingLayer2. The merge is available only if the hardware supports the related feature.
	-sub2convadd:0 1	-sub2convadd:1	Specifies whether to convert subtract into ConvolutionReluPoolingLayer2. Set this option to one of the following values: 0: does not convert subtract. 1: converts subtract into ConvolutionReluPoolingLayer2. The conversion is available only if the hardware supports the related feature.

5.2 Use Cases

This section provides several common use cases for the environment variables described in [Section 5.1, Environment Variable](#). Example system logs are also provided for your reference.

5.2.1 Disabling OVXLIB Log Printing

To disable OVXLIB log printing, set `VSI_NN_LOG_LEVEL=0`.

An example of the network execution result returned when OVXLIB log printing is disabled is shown as follows:

```
1. Create Neural Network: 7ms or 7716us
2. Verify...
3. Verify Graph: 2180ms or 2180954us
4. Start run graph [1] times...
5. Run the 1 time: 18ms or 18650us
6. vxProcessGraph execution time:
7. Total 18ms or 18676us
8. Average 18.68ms or 18676.00us
9. --- Top5 ---
10. 0: 0.999512
11. 5: 0.000158
12. 6: 0.000140
13. 2: 0.000096
14. 1: 0.000000
```

5.2.2 Enabling OVXLIB Log Printing

To enable OVXLIB log printing, set `VSI_NN_LOG_LEVEL` to a value from 1 to 4.

An example of the network execution result returned when OVXLIB log printing of the warning level is enabled (`VSI_NN_LOG_LEVEL=2`) is shown as follows:

```
1. D [setup_node:434]Setup node id[0] uid[1] op[CONV2D]
2. D [print_tensor:138]in(0) : id[ 0] vtl[0] const[0] shape[ 28, 28, 1, 1 ] fmt[u8 ] qnt[ASM zp
   = 0, scale=0.003922]
3. D [print_tensor:138]in(1) : id[ 2] vtl[0] const[1] shape[ 5, 5, 1, 20 ] fmt[u8 ] qnt[ASM zp
   =119, scale=0.003362]
4. D [print_tensor:138]in(2) : id[ 3] vtl[0] const[1] shape[ 20 ] fmt[i32] qnt[ASM zp
   = 0, scale=0.000013]
5. D [print_tensor:138]out(0): id[ 10] vtl[1] const[0] shape[ 24, 24, 20, 1 ] fmt[u8 ] qnt[ASM zp
   =140, scale=0.019360]
6. D [setup_node:434]Setup node id[1] uid[2] op[POOL]
7. D [print_tensor:138]in(0) : id[ 10] vtl[1] const[0] shape[ 24, 24, 20, 1 ] fmt[u8 ] qnt[ASM zp
   =140, scale=0.019360]
8. D [print_tensor:138]out(0): id[ 11] vtl[1] const[0] shape[ 12, 12, 20, 1 ] fmt[u8 ] qnt[ASM zp
   =140, scale=0.019360]
9. D [setup_node:434]Setup node id[2] uid[3] op[CONV2D]
10. D [print_tensor:138]in(0) : id[ 11] vtl[1] const[0] shape[ 12, 12, 20, 1 ] fmt[u8 ] qnt[ASM zp
   =140, scale=0.019360]
11. D [print_tensor:138]in(1) : id[ 4] vtl[0] const[1] shape[ 5, 5, 20, 50 ] fmt[u8 ] qnt[ASM zp
   =128, scale=0.001148]
```

```

12. D [print_tensor:138]in(2) : id[ 5] vtl[0] const[1] shape[ 50                ] fmt[i32] qnt[ASM zp
    = 0, scale=0.000022]
13. D [print_tensor:138]out(0): id[ 12] vtl[1] const[0] shape[ 8, 8, 50, 1      ] fmt[u8 ] qnt[ASM zp
    =141, scale=0.040921]
14. D [setup_node:434]Setup node id[3] uid[4] op[POOL]
15. D [print_tensor:138]in(0) : id[ 12] vtl[1] const[0] shape[ 8, 8, 50, 1      ] fmt[u8 ] qnt[ASM zp
    =141, scale=0.040921]
16. D [print_tensor:138]out(0): id[ 13] vtl[1] const[0] shape[ 4, 4, 50, 1      ] fmt[u8 ] qnt[ASM zp
    =141, scale=0.040921]
17. D [setup_node:434]Setup node id[4] uid[5] op[FCL]
18. D [print_tensor:138]in(0) : id[ 13] vtl[1] const[0] shape[ 4, 4, 50, 1      ] fmt[u8 ] qnt[ASM zp
    =141, scale=0.040921]
19. D [print_tensor:138]in(1) : id[  6] vtl[0] const[1] shape[ 800, 500         ] fmt[u8 ] qnt[ASM zp
    =130, scale=0.000735]
20. D [print_tensor:138]in(2) : id[  7] vtl[0] const[1] shape[ 500                ] fmt[i32] qnt[ASM zp
    = 0, scale=0.000030]
21. D [print_tensor:138]out(0): id[ 14] vtl[1] const[0] shape[ 500, 1            ] fmt[u8 ] qnt[ASM zp
    = 0, scale=0.019997]
22. D [setup_node:434]Setup node id[5] uid[6] op[RELU]
23. D [print_tensor:138]in(0) : id[ 14] vtl[1] const[0] shape[ 500, 1            ] fmt[u8 ] qnt[ASM zp
    = 0, scale=0.019997]
24. D [print_tensor:138]out(0): id[ 15] vtl[1] const[0] shape[ 500, 1            ] fmt[u8 ] qnt[ASM zp
    = 0, scale=0.019997]
25. D [setup_node:434]Setup node id[6] uid[7] op[FCL]
26. D [print_tensor:138]in(0) : id[ 15] vtl[1] const[0] shape[ 500, 1            ] fmt[u8 ] qnt[ASM zp
    = 0, scale=0.019997]
27. D [print_tensor:138]in(1) : id[  8] vtl[0] const[1] shape[ 500, 10          ] fmt[u8 ] qnt[ASM zp
    =135, scale=0.001580]
28. D [print_tensor:138]in(2) : id[  9] vtl[0] const[1] shape[ 10                ] fmt[i32] qnt[ASM zp
    = 0, scale=0.000032]
29. D [print_tensor:138]out(0): id[ 16] vtl[1] const[0] shape[ 10, 1             ] fmt[u8 ] qnt[ASM zp
    = 80, scale=0.062750]
30. D [setup_node:434]Setup node id[7] uid[8] op[SOFTMAX]
31. D [print_tensor:138]in(0) : id[ 16] vtl[1] const[0] shape[ 10, 1             ] fmt[u8 ] qnt[ASM zp
    = 80, scale=0.062750]
32. D [print_tensor:138]out(0): id[  1] vtl[0] const[0] shape[ 10, 1             ] fmt[f16] qnt[NONE]

33. D [optimize_node:378]Backward optimize neural network
34. D [optimize_node:385]Forward optimize neural network
35. D [vsi_nn_internal_optimize_node:614]Optimize node uid[8] sub_uid[1] op[SOFTMAX_INTERNAL]
36. I [compute_node:326]Create vx node
37. D [compute_node:349]Instance node[0] "CONV2D" ...
38. D [compute_node:349]Instance node[1] "POOL" ...
39. D [compute_node:349]Instance node[2] "CONV2D" ...
40. D [compute_node:349]Instance node[3] "POOL" ...
41. D [compute_node:349]Instance node[4] "FCL" ...
42. D [compute_node:349]Instance node[5] "RELU" ...
43. D [compute_node:349]Instance node[6] "FCL" ...
44. D [compute_node:349]Instance node[7] "SOFTMAX" ...
45. D [vsi_nn_internal_compute_node:587]Compute node uid[8] sub_uid[1] op[SOFTMAX_INTERNAL]

```

5.2.3 Running a Network Multiple Times

To run a network multiple times, set `VNN_LOOP_TIME` to an integer greater than 1.

Note: To reduce the network loading time, it is recommended that you enable graph caching by setting `VIV_VX_ENABLE_CACHE_GRAPH_BINARY=1`.

An example of the network execution result returned when the network is run 10 times is shown as follows:

```
1. Create Neural Network: 6ms or 6825us
2. Verify...
3. Verify Graph: 2186ms or 2186652us
4. Start run graph [10] times...
5. Run the 1 time: 18ms or 18560us
6. Run the 2 time: 18ms or 18328us
7. Run the 3 time: 18ms or 18416us
8. Run the 4 time: 18ms or 18201us
9. Run the 5 time: 18ms or 18446us
10. Run the 6 time: 18ms or 18412us
11. Run the 7 time: 18ms or 18412us
12. Run the 8 time: 18ms or 18296us
13. Run the 9 time: 18ms or 18469us
14. Run the 10 time: 18ms or 18738us
15. vxProcessGraph execution time:
16. Total 184ms or 184512us
17. Average 18.45ms or 18451.20us
18. --- Top5 ---
19. 0: 0.999512
20. 5: 0.000158
21. 6: 0.000140
22. 2: 0.000096
23. 1: 0.000000
```

Calculating Network Execution Time

You can use the average network execution time provided in the Line 17 of the log to calculate the number of VIP cycles consumed to run the network as follows:

$$\text{VIP cycle count} = \text{VIP core main frequency} \times \text{Average execution time}$$

The calculation result includes interrupt responses. If you need an accurate measurement, calculate the network execution time as described in [Section 5.2.5, Profiling a Network](#).

5.2.4 Enabling Driver Log Printing

To print the driver log, set `VIV_VX_DEBUG_LEVEL=1`.

An example of the returned driver log is shown as follows:

```
1. -----Begin VerifyTiling -----
2. AXI-SRAM = 0 Bytes VIP-SRAM = 260096 Bytes SWTILING_PHASE_FEATURES[1, 1, 1]
3. 0 NN [( 28 28 1 1, 784, 0x0x1fcd9c0(0x0x1fcd9c0, 0x(nil)) -> 12 12 20 1, 28
80, 0x0x1fd46e0(0x0x1fd46e0, 0x(nil))) k(5 5 1, 640) pad(0 0) pool(2 2, 2 2)] C[ 1]
4. 1 NN [( 12 12 20 1, 2880, 0x0x1fd46e0(0x0x1fd46e0, 0x(nil)) -> 4 4 50 1, 8
00, 0x0x1fd9580(0x0x1fd9580, 0x(nil))) k(5 5 20, 26752) pad(0 0) pool(2 2, 2 2)] P[ 0] C[ 2
]
```

```

5. 2 TP [( 800 1 1 1, 800, 0x0x1fd9580(0x0x1fd9580, 0x(nil)) -> 500 1 1 1, 5
00, 0x0x1fdc650(0x0x1fdc650, 0x(nil))) k(0 0 0, 0) pad(0 0) pool(0 0, 1 1)] P[ 1] C[ 3
]
6. 3 TP [( 500 1 1 1, 500, 0x0x1fdc650(0x0x1fdc650, 0x(nil)) -> 10 1 1 1,
10, 0x0x1fdd6a0(0x0x1fdd6a0, 0x(nil))) k(0 0 0, 0) pad(0 0) pool(0 0, 1 1)] P[ 2] C[ 4
]
7. 4 SH [( 10 1 1 1, 10, 0x0x1fdd6a0(0x0x1fdd6a0, 0x(nil)) -> 10 1 1 1,
20, 0x0x1fce320(0x0x1fce320, 0x(nil))) k(0 0 0, 0) pad(0 0) pool(0 0, 1 1)] P[ 3]
8.
9. Detected Segments
10. AB_VS (0 - 4)
11. ===== Block [0 - 4] =====
12. 0 NN DD -> VS [( 784, 2944), IC( 384), KC( 1024)]
13. 1 NN VS -> VS [( 2944, 896), IC( 0), KC( 31360)]
14. 2 TP VS -> VS [( 896, 512), IC( 0), KC( 0)]
15. 3 TP VS -> VS [( 512, 128), IC( 0), KC( 0)]
16. 4 SH VS -> DD [( 128, 20), IC( 0), KC( 0)]
17. -----
18. Segment AB (0 - 4)
19.
20. AXISRAM: Peak used 0 0.000000% VIPSRAM: Peak used 35200 13.533464%
21. ===== Block [0 - 4] SUCCEED =====
22.
23. id IN [ x y w h ] OUT [ x y w h ] (tx, ty, kpc) (ic, kc, kc/ks, ks/eks, kernel_type)
24. 0 NN DD 0x(nil) [ 0 0 28 28] -> VS 0x0x400800 [ 0 0 12 12] (
24, 8, 5) ( 384, 1024, 100.000000%, 160.000000%, DD)
25. 1 NN VS 0x0x400800 [ 0 0 12 12] -> VS 0x0x408e00 [ 0 0 4 4]
( 8, 8, 13) ( 0, 31360, 100.000000%, 117.224884%, DD)
26. 2 TP VS 0x0x408e00 [ 0 0 800 1] -> VS 0x0x400800 [ 0 0 500 1]
( 0, 0, 0) ( 0, 0, 0.000000%, 0.000000%, NONE)
27. 3 TP VS 0x0x400800 [ 0 0 500 1] -> VS 0x0x409100 [ 0 0 10 1]
( 0, 0, 0) ( 0, 0, 0.000000%, 0.000000%, NONE)
28. 4 SH VS 0x0x409100 [ 0 0 10 1] -> DD 0x(nil) [ 0 0 10 1] (
0, 0, 0) ( 0, 0, 0.000000%, 0.000000%, NONE)
29.
30. PreLoadWeightBiases = 0 -nan%
31. -----End VerifyTiling -----

```

5.2.5 Profiling a Network

To print the VIP performance log for an entire network, set **VIV_VX_PROFILE=1**, **VIV_VX_DEBUG_LEVEL=1**, and **CNN_PERF=0**.

An example of the VIP performance log for a network is shown as follows. The log fields are detailed in [Table 14](#).

```

1. [ 1] TOTAL_READ_BANDWIDTH (MByte): 0.452833
2. [ 2] TOTAL_WRITE_BANDWIDTH (MByte): 0.000061
3. [ 3] AXI_READ_BANDWIDTH (MByte): 0.000000
4. [ 4] AXI_WRITE_BANDWIDTH (MByte): 0.000000
5. [ 5] DDR_READ_BANDWIDTH (MByte): 0.452833
6. [ 6] DDR_WRITE_BANDWIDTH (MByte): 0.000061
7. [ 7] GPUTOTALCYCLES: 272243
8. [ 8] GPUIDLECYCLES: 42092

```


Table 14. Network/Layer Profile

Line No.	Description
1	The total read bandwidth, including the AXI bandwidth and the DDR bandwidth.
2	The total write bandwidth, including the AXI bandwidth and the DDR bandwidth.
3	The AXI read bandwidth.
4	The AXI write bandwidth.
5	The DDR read bandwidth.
6	The DDR write bandwidth.
7	The total number of VIP cycles.
8	The number of idle VIP cycles.

Note: For a layer, multiple profiles may be printed because of tiling. In this case, to measure the bandwidth and the VIP cycles consumed by the layer, sum up the bandwidth and VIP cycles in each profile.

Calculating Network Execution Time

You can use the total number of VIP cycles and the number of idle VIP cycles provided in Line 7 and Line 8 of the log to calculate the number of VIP cycles consumed to run the network or layer as follows:

$$\text{VIP cycle count} = \text{Total number of VIP cycles} - \text{Number of idle VIP cycles}$$

5.2.6 Profiling Layers

To print the per-layer VIP performance log, set **VIV_VX_PROFILE=1**, **VIV_VX_DEBUG_LEVEL=1**, and **CNN_PERF=1**.

An example of the per-layer VIP performance log is shown as follows. The log fields are detailed in [Table 14](#).

```

1. layer id: 1 layer name:ConvolutionReluPoolingLayer2 operation[0]:VXNNE_OPERATOR_CONVOLUTION target
   :VXNNE_OPERATION_TARGET_NN.
2. uid: 2
3. op_abs_id: 0
4. execution time: 4070 us
5. [ 1] TOTAL_READ_BANDWIDTH (MByte): 0.002457
6. [ 2] TOTAL_WRITE_BANDWIDTH (MByte): 0.000000
7. [ 3] AXI_READ_BANDWIDTH (MByte): 0.000000
8. [ 4] AXI_WRITE_BANDWIDTH (MByte): 0.000000
9. [ 5] DDR_READ_BANDWIDTH (MByte): 0.002457
10. [ 6] DDR_WRITE_BANDWIDTH (MByte): 0.000000
11. [ 7] GPUTOTALCYCLES: 43003
12. [ 8] GPUIDLECYCLES: 41284

```

Note: The layer UID in Line 2 is originated from ACUITY. The UID is also returned by the network profiling function of VivanteIDE. With the UID, you can compare the actual hardware performance with the VivanteIDE simulated performance.

5.2.7 Profiling Memory Usage

To print the memory usage information of an application, set `VIV_MEMORY_PROFILE=1`.

An example of the memory usage log is shown as follows. It prints both the system memory usage and the video memory usage. The statistics about allocated memory are measured in bytes.

```

1. [ 41] ***** Memory Profile Info Dump *****
2. [ 42] system memory:
3. [ 43]   current allocation      : 0
4. [ 44]   maximum allocation     : 38435795
5. [ 45]   total allocation       : 1226619259
6. [ 46]   total free            : 1226619259
7. [ 47]   allocation count      : 396654
8. [ 48]   free count            : 396654
9. [ 49]
10. [ 50] gpu memory:
11. [ 51]   current allocation      : 0
12. [ 52]   maximum allocation     : 1582368
13. [ 53]   total allocation       : 1582496
14. [ 54]   total free            : 1582496
15. [ 55]   allocation count      : 229
16. [ 56]   free count            : 229
17. [ 57] ***** end of Memory Profile Info Dump *****

```

5.2.8 Specifying Multiple Optimization Options

The following command shows an example of how to specify multiple optimization options:

```
export VIV_VX_ENABLE_GRAPH_TRANSFORM=-Dump-tensorAdd:0
```

5.2.9 Configuring Multi-VIP Scenarios

In multi-VIP scenarios, a system has multiple VIP cores instantiated. To configure a multi-VIP scenario, use one set of the following environment variables:

- `VIV_VX_ENABLE_MULTIVIP_COMBINED`
- `VIV_MGPU_AFFINITY` and `VIV_OVX_USE_MULTI_DEVICE`

The following uses `VIV_MGPU_AFFINITY` and `VIV_OVX_USE_MULTI_DEVICE` as an example to describe how to configure a multi-VIP scenario:

- If you need to enable all the VIP cores to work on the same device, set `VIV_MGPU_AFFINITY=0`.
- If you need to separate the VIP cores to different devices, set `VIV_MGPU_AFFINITY=1:0`.

To specify the number of VIP cores that correspond to each device, use `VIV_OVX_USE_MULTI_DEVICE`.

For example, there are eight VIP cores in a system. To enable the application to use up to eight devices, run the following commands:

```
export VIV_MGPU_AFFINITY=1:0
export VIV_OVX_USE_MULTI_DEVICE=1:1
```

Then, the devices, each with a VIP core, can run different graphs in parallel. You can specify the graph which each device runs as follows:

```
args[i].deviceId = i;  
args[i].graph = graph[i];  
vxSetGraphAttribute(args[i].graph[i], VX_GRAPH_DEVICE_INDEX_VIV, &args[i].deviceId,  
sizeof(args[i].deviceId))
```



6 VIPLite Driver

This chapter describes how to run networks and create applications with Vivante VIPLite driver.

For more information about Vivante VIPLite driver, see the reference documents listed in the [References](#) section.

6.1 VIPLite Driver Packages

Unified driver packages include:

- Vivante_VIPLite_Src_drv_{version}.org.tgz
- Vivante_VIPLite_Src_drv_{customer}_platform_{version}.org.tgz
- Vivante_VIPLite_Src_drv_freeRTOS_{version}.org.tgz
- Vivante_VIPLite_Src_{customer}_tst_{version}.org.tgz

There may be other additional add-ons or patches when needed.

6.2 Building the VIPLite Driver

Before networks can run with Vivante VIPLite driver, you need to build it.

Procedure

1. Porting the driver by referencing *Vivante Programming: VIPLite Driver Porting Guide*.
2. Extract all the driver packages to the same directory as follows:

```
tar xzvf Vivante_VIPLite_Src_drv_{version}.org.tgz  
tar xzvf Vivante_VIPLite_Src_drv_{customer}_platform_{version}.org.tgz
```
3. (Optional) Apply patches if any.
4. Modify the configurations in the `build_drv_linux.sh` file with your tool chain.
5. Run the `build_drv_linux.sh` file as follows:

```
./build_drv_linux.sh vim3 install
```

6.3 Configuring Network Profiling for VIPLite Driver

To configure network profiling for the Vivante VIPLite driver on hardware platforms, you need to modify relevant macro definitions in the `hal/inc/gc_vip_common.h` file.

Procedure

In the `hal/inc/gc_vip_common.h` file, set the macros listed in the following table.

Table 15. Macro Definitions

Macro	Description
<code>vpmdENABLE_DEBUG_LOG</code>	Specifies the log levels to be printed. Set this macro to one of the following values: • 0: does not print logs. • 1: prints logs of the error level. • 2: prints logs of the error and warning levels. • 3: prints logs of the error, warning, and information levels. • 4: prints logs of the error, warning, information, and debug levels.
<code>vpmdENABLE_CNN_PROFILING</code>	Specifies whether to return the operation execution time. Set this macro to one of the following values: • 0: does not return the operation execution time. • 1: returns the execution time of each operation. For sample logs, see Section 6.3.2, Layer Profile.
<code>vpmdENABLE_BW_PROFILING</code>	Specifies whether to return the bandwidth information. Set this macro to one of the following values: • 0: does not return the bandwidth information. • 1: returns bandwidth information of each operation after each command is submitted to the buffer. For sample logs, see Section 6.3.1, Network Profile and Section 6.3.2, Layer Profile.
<code>vpmdENABLE_MEMORY_PROFILING</code>	Specifies whether to return the memory usage information. Set this macro to one of the following values: • 0: does not the memory usage information. • 1: returns the system memory usage and video memory usage. For sample logs, see Section 6.3.3, Memory Footprint.

Macro	Description
<code>vpmdENABLE_APP_PROFILING</code>	Specifies whether to return the time that the inference takes. Set this macro to one of the following values: 0: does not return the time information. 1: returns the time that the inference takes. Both the time in microseconds and that in VIP cycles are returned. For sample logs, see Section 6.3.4, Inference Time.
<code>vpmdPOWER_OFF_TIMEOUT</code>	Specifies the timeout period for the VIP hardware to stay in idle state before powering off. The VIP hardware automatically powers off from the idle state when a specified timeout is reached. Set this macro as follows: - To enable the automatic power-off feature, set the macro to a value greater than or equal to 10. The value is measured in milliseconds. - To disable the automatic power-off feature, set the macro to 0. - Other values are invalid. Note: VIPLite power management does not support triggering hardware by using a wait-link loop.
<code>vpmdENABLE_CAPTURE</code>	Specifies whether to dump a hardware capture for hardware simulation. Set this macro to one of the following values: 0: does not dump a hardware capture. 1: dumps a capture of the states and command buffer. 2: dumps a capture of the states, command buffer, input, output, and coefficients. The dumped capture is saved to the file <code>viplite_hal_capture_xxx_xx_xxxx.log</code>.

6.3.1 Network Profile

When `vpmdENABLE_BW_PROFILING` is set to 1, the network profile shows on the screen as follows:

```
Total read bandwidth (Byte): 3036
Total write bandwidth (Byte): 1953
AXI read bandwidth (Byte): 0
AXI write bandwidth (Byte): 0
DDR read bandwidth (Byte): 3036
DDR write bandwidth (Byte): 1953
VIP total cycles: 130230
VIP total idle cycles: 4457
```

6.3.2 Layer Profile

When `vpmdENABLE_CNN_PROFILING` and `vpmdENABLE_BW_PROFILING` are set to 1, the layer profiles show on the screen. A layer profile example is as follows. Key log fields are detailed in [Table 16](#).

```
layer id: 54, uid: 207, op id: 58, layer name: FullyConnectedReluLayer, operation target: TP
execution time: 20978 us
@{
  tp_layer_id: 1
  tp_total_busy_cycle: 209663
  tp_total_read_bandwidth_cache: 719872
  tp_total_write_bandwidth_cache: 1000
  tp_total_read_bandwidth_sram: 0
  tp_total_write_bandwidth_sram: 0
  tp_total_read_bandwidth_ocb: 0
  tp_total_write_bandwidth_ocb: 0
  tp_fc_pix_count: 5003
  tp_fc_zero_skip_count: 1804
  tp_fc_coef_count: 809976
  tp_fc_coef_zero_count: 22050
  tp_total_idle_cycle_core[0]: 8162
}@
```

Table 16. Key Fields in a Layer Profile

Field Name	Description
<code>tp_total_busy_cycle</code>	The number of busy VIP cycles.
<code>tp_total_read_bandwidth_cache</code>	The DDR read bandwidth.
<code>tp_total_write_bandwidth_cache</code>	The DDR write bandwidth.

6.3.3 Memory Footprint

When `vpmdENABLE_MEMORY_PROFILING` is set to 1, the memory footprint shows on the screen as follows:

```
===== Memory Statistics =====
Current Video Memory Used: 0 bytes.
Video Peak Memory Used: 12211888 bytes.
Video Memory Alloc Count: 68 times.
Video Memory Free Count: 68 times.

Current System Memory Used: 0 bytes.
System Peak Memory Used: 36788 bytes.
System Memory Alloc Count: 18 times.
System Memory Free Count: 18 times.
===== End =====
```


6.3.4 Inference Time

When `vpmdENABLE_APP_PROFILING` is set to 1, the inference time shows on the screen as follows:

```
profile inference time=247751us, cycle=2416220
```

6.4 Running Networks and Creating Applications

For information about how to test an NBG project and create an application, see the `vpmd_run` file in the provided project.

7 NBInfo

VeriSilicon NBInfo is a tool used to parse network binary graph (NBG) files whose extension is .nb. It depends on the NBG Parser library that is used to parse the NBG format.

NBInfo can interpret NBG files to show detailed information, including the input, output, layers, and operations, in the NBG file. You can select the information to be shown by using the following options listed in the following table.

Table 17. NBInfo Options

Option	Description
-a	Prints all information in the NBG file.
-b	Prints the brief summary of the NBG file, including the version, hardware target, and AXI SRAM size.
-l	Prints the layer table in the NBG file.
-o	Prints the operation table in the NBG file.
-in	Prints the input table in the NBG file.
-out	Prints the output table in the NBG file.
-m	Prints memory information in the NBG file, including the user heap size and video memory size.
-d	Prints the detailed memory information. It is a sub-option of -m.
-f	Profiles the video memory in flash. It is a sub-option of -m.
-h	Prints the usage text.

7.1 Obtaining the Brief Summary of an NBG File

You can obtain the major information of an NBG file by running the following command:

```
nbinfo -b <NBG file name>
```

For example, the following script shows the output of the `nbinfo -b network_binary.nb` command:

```
Verisilicon NBInfo version 1.1.4, version=0x00010104

nbg file name network_binary.nb
*****
Overall Info
*****
Network Name:                dummy_network_name
Version:                     0x1000f
Target:                      0xba
Core Count:                  1
AXI SRAM Size:               0
VIP SRAM Size:               0x40000
Memory Pool Size (bytes):    2850304
Memory Pool Alignment:      64
Layer Count:                 282
Operation Count:             1805
```

```
Input Count: 5
Output Count: 5
*****
```

7.2 Querying the Memory Overhead of an NBG File

You can query the memory overhead of an NBG file by running the following command:

```
nbinfo -m -d <NBG file name>
```

For example, the following script shows the output of the `nbinfo -m -d network_binary.nb` command:

```
Verisilicon NBInfo version 1.1.4, version=0x00010104

nbg file name network_binary.nb

*****
Input and memory_pool overlap info
*****
input[0] layer[0] name=ConvolutionReluPoolingLayer2, output into memory pool
Summary:
    not support share buffer between input and memory pool.
*****

*****
Memory Info
*****
Total Video Memory (bytes): 499968
-- Memory Pool (bytes): 4864
-- Total Input Memory (bytes): 832
-- Total Output Memory (bytes): 64
-- Total Operation Memory (bytes): 473664
-- Total Read Only Memory (bytes): 471168
-- Total Command buffer (bytes): 5312
-- Total Load States (bytes): 3776
-- Total NN and TP instruction (bytes): 512
-- Total PPU instruction (bytes): 1024
-- Dynamic Input or Output (bytes): 0
-- Video memory heap node reserved (bytes): 20480
*****
Total System Memory (bytes): 4348
-- NBG header Size (bytes): 2868
-- Allocate memory size (bytes): 1480
*****
```

7.3 Querying the Input Parameters in an NBG File

You can query the input parameters in an NBG file by running the following command:

```
nbinfo -in <NBG file name>
```

For example, the following script shows the output of the `./nbinfo -in network_binary.nb` command:

```
*****
Input Table
*****
Dim Count:                                2
Size of Dim[0]:                           1296
Size of Dim[1]:                           432
Size of Dim[2]:                           1
Size of Dim[3]:                           1
Data Format:                               VIP_BUFFER_FORMAT_UINT8
Data Type:                                VIP_BUFFER_TYPE_IMAGE
Quantization Format:                       VIP_BUFFER_QUANTIZE_NONE
Fixed Point Pos:                          0
TF Scale:                                 0.000000
TF Zeropoint:                             0
Memory Size (bytes):                      559872
*****
```

7.4 Querying the Output Parameters in an NBG File

You can query the output parameters in an NBG file by running the following command:

```
nbinfo -out <NBG file name>
```

For example, the following script shows the output of the `./nbinfo -out network_binary.nb` command:

```
*****
Output Table
*****
Dim Count:                                2
Size of Dim[0]:                           7668
Size of Dim[1]:                           1
Size of Dim[2]:                           1
Size of Dim[3]:                           1
Data Format:                               VIP_BUFFER_FORMAT_UINT8
Data Type:                                VIP_BUFFER_TYPE_TENSOR
Quantization Format:                       VIP_BUFFER_QUANTIZE_TF_ASYMM
Fixed Point Pos:                          0
TF Scale:                                 0.124952
TF Zeropoint:                             169
Memory Size (bytes):                      7680
*****
Dim Count:                                2
Size of Dim[0]:                           40257
Size of Dim[1]:                           1
Size of Dim[2]:                           1
Size of Dim[3]:                           1
Data Format:                               VIP_BUFFER_FORMAT_FP16
Data Type:                                VIP_BUFFER_TYPE_TENSOR
Quantization Format:                       VIP_BUFFER_QUANTIZE_NONE
*****
```

```
Fixed Point Pos: 0
TF Scale: 1.000000
TF Zeropoint: 0
Memory Size (bytes): 80640
*****
```



Document Revision History

This section describes top level differences between document revisions.

Note: This document is not usually updated for each patch or minor revision. The information in this document tends to be stable across a revision (*nnn*) series.

Document Revision	Date	Compatibility	Description
0.81	2023-03-31	- ACUITY Toolkit 6.18.0 and above - VivanteIDE 5.8.1 and above - Unified Driver 6.4.14 and above - VIPLite Driver 1.12.0 and above	- Added Section 3.2, Using VivanteIDE Command Line Tool, and restructured Chapter 3, VivanteIDE. - Separated Chapter 5, Environment Variables from Chapter 4, Unified Driver. Section 2.1, ACUITY Toolkit Packages: Added the restriction that ACUITY Toolkit packages cannot be decompressed on Windows. Section 3.1.3, Importing an OpenVX Project: Added the prerequisite that the OpenVX project must be located in a different directory from the workspace. Section 4.2.1, Building the Driver: Added build options and commands for hardware that supports 40-bit virtual address space. Section 4.6.2, Using Image/Tensor Handles: Added OVXLIB APIs for tensor handles. Section 4.6.2.2, Creating Images/Tensors from Handles: Updated the sample script for creating a tensor from a handle. Section 4.6.2.4, Example: Updated the sample script.
0.80	2022-09-09	- ACUITY Toolkit 6.9.0 and above - VivanteIDE 5.7.1 and above - Unified Driver 6.4.11 and above - VIPLite Driver 1.8.0 and above	Initial release.